How errors survive depth in cumulant propagation

A closed-form survival law, where the remaining error is made, and derived corrections on a sampling-free estimator

megan_park — ARC White-Box Estimation Challenge 2026, Phase 2 · linked submission #337350

---

0. Summary

What this is. Submission #337350 (megan_park) predicts the mean activation of every neuron of a random 1024-wide, 16-layer ReLU network from its weights alone. At prediction time it draws no samples. Each layer pushes the means, the covariance and low-rank third cumulants through the weights, then applies the ReLU through Gaussian (Hermite–Wick) coefficients and our derived correction terms.

- It scored 1.918e-9 on the official public networks, with 100/100 networks completed.
- Its final-layer MSE is 84× below Monte Carlo spending the whole budget, at 14 % of that budget. At equal compute that is 590×.
- Prizes follow a rescoring on new networks.
- Code, checks, ablation receipts and tool reproductions: https://github.com/Meganpark980320/arc-whitebox-p2-337350.

What we contribute to ARC's question of why its methods break down with depth.

1. *How errors fade (§2).* How much of a layer's error reaches the output is fixed by the network: $S(\ell) = \prod_{k>\ell}(1 - \arccos(c_k)/\pi)$, the slope of the mean-field correlation map.
   - With no fitted parameter it matches our engine's error ledger within 1–4 %, and it predicts how the final error splits over the layers where it was made.
   - Forgetting is polynomial, so the number of layers whose errors reach the output grows linearly with depth: 6 at depth 16, 19 at 64.
   - Shared scale and bias errors barely fade.
2. *Where the remaining error is made (§3).* The error is made in the carried state.
   - Truth oracles cut the MSE by 29 % in the covariance (layers 11–14) and by 11 % in the κ3 legs (layers 10–14), on tuning networks.
   - Each layer refills the κ3 error mostly (three quarters to four fifths, on two networks) from κ4 that is old and converted from κ3. No memoryless closure can supply it.
   - A fix written into one layer's slot does not enter the state.
3. *Derived corrections (§4).* Seven terms from ARC's expansion, folded into products the engine already computes.
   - Feedback-birth is the largest: switching it off alone nearly quadruples the MSE, and its derived strength ½ beats its inherited ¼ by 17 %.
   - O1 (−0.87 %) and ACC (−0.52 %) were official single changes.

What it means, and what we found in the tools (§6, Appendix H).

- *Depth.* Depth hurts through slow forgetting: to keep the output error fixed from depth 16 to 64, each layer's new error must shrink about 3×. Shared scale and bias errors are the dangerous ones, and the missing piece is memory (κ4 converted from old κ3).
- *Fits.* The fitted factor of about 1.8 is a measured target for theory. §6.4 lists five open problems.
- *Tools and scoring.* Two flopscope bugs pass every in-process test and fail only in grading: a tuple `axis` in client–server mode, and einsum's cost analysis allocating a float64 buffer the size of each operand (with a verified patch). We also report two smaller tool issues and three observations on the scoring rules, each reproduced on the current releases with scripts in the repository.

Honest accounting (§§1.3, 5). The score rests substantially on fitted parts.

- *All switchable fits off.* This covers the learned corrections, the learned κ4 parts, the tuned strengths and gains, and the inherited fitted κ4 constant λ. Switching them all off raises the MSE by 78–82 % on two network sets.
- *What remains is still strong.* The remaining engine still keeps 410 fitted constants that have no switch, and it is 330× better than Monte Carlo at equal compute. For comparison: ARC's reference chain is 118×, our starting engine with the same physical λ is 142×, and the shipped build is 594×.
- *Where the gain came from.* Accuracy and compute each carry half of our 62 % gain over the public starting point.

Who did what (§8). The participant set the research direction and proposed most of the ideas. AI agents (Claude via Claude Code, and OpenAI Codex) implemented them, ran the checks and drafted this text.

1. Task, method, and what is derived versus fitted

1.1 Task and claim

- **Task.** From the weights of a bias-free ReLU network (width n = 1024, 16 hidden layers ℓ = 0…15, input $\boldsymbol{x} \sim \mathcal{N}(0, I_n)$, weights $\mathcal{N}(0, 2/n)$), return every neuron's mean activation at every layer. A network scores its final-layer MSE against Monte Carlo truth times max(0.1, F/2^41), with F the FLOPs billed by flopscope (1 GF = 10^9 FLOPs). The leaderboard averages networks.
- **What ARC asks.** Can one predict what a network does from its weights alone? Error at fixed compute is compared with Monte Carlo sampling; any method is allowed.
- **Our claim.** We draw no samples at predict time (§1.4). #337350 scores 1.918e-9 on the 50 public networks. Its MSE is 84× below full-budget Monte Carlo at 14 % of the budget, and 590× at equal compute [official bundle]. Fitted parts carry part of this (§1.3, §5).
- **Harness limits:** setup 5 s per worker with import, else the run fails; predict 120 s per network; residual client time 0.4 s per network, else the answer is zeroed; 8 GiB address space per worker.

Sources: `docs/concepts/problem-setup.md`; `.venv/lib/python3.10/site-packages/whestbench/budget.py`; `experiments/claude_liaison_local_20261017/official_eval/eval_337350_C_fin4_cf4g065_completed.json`.

1.2 Prior work and the engine

- *ARC's cumulant propagation* (arXiv 2605.05179; `alignment-research-center/mlp_cumulant_propagation`) propagates per-neuron means and low-order joint cumulants through Hermite expansions of the ReLU.
- *504aldo's V29 engine* (`github.com/504aldo/whest-p2-cumulant-k3`) makes this affordable. Each *source*, a κ3 born at one layer, keeps two low-rank *legs*, transported as $\boldsymbol{A} \leftarrow \boldsymbol{W}_\ell^\top \boldsymbol{D}_q \boldsymbol{A}$. κ4 is regenerated without memory as G = diag(dG) + λ_ℓ C_off, with fitted λ_ℓ. Pruned Wick tables apply the ReLU; products are Strassen–Winograd recursions.
- *corpaci's #332675* (`github.com/corpaci/ARCwhitebox`), our starting point, ports V29 with corpaci's Strassen recursion to the Phase 2 harness: score 5.0495e-9, MSE 2.188e-8, 23.1 % of the budget [official bundle].
- *The engine carries* means m, variances v, gate statistics q, d, the covariance, each source's legs A, P, the represented (2,1) slice D21 (left *slabs* times A^T, P^T) and the κ4 diagonal.
- *Our learned corrector*, a 171-64-64-1 MLP trained on Monte Carlo labels, corrects the mean inside the loop at layers 6–15.

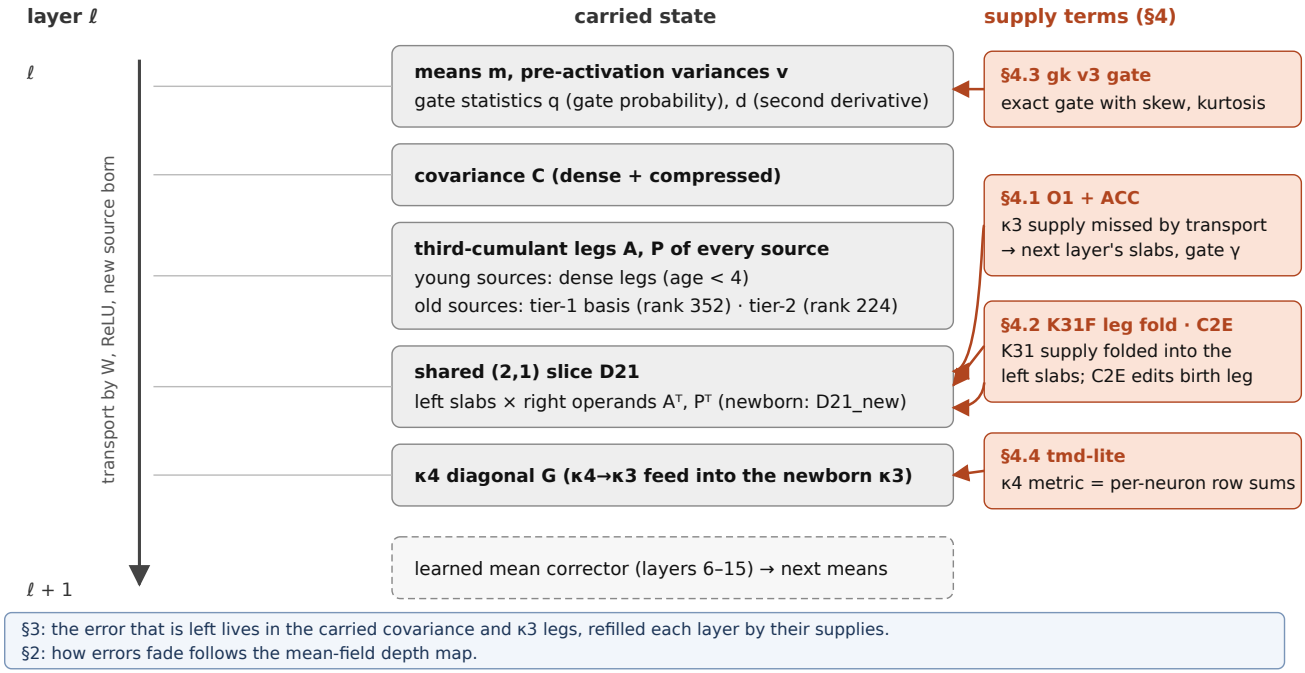


Figure 1. What the engine carries from layer ℓ to $\ell + 1$ (centre), and where our derived supply terms enter (orange, right): the gk v3 gate on the gate statistics; O1 and ACC on the next layer's D21 slabs; the K31F leg fold on the D21 slabs and C2E on the newborn leg; and tmd-lite on the κ4 metric of the κ4→κ3 feed (all §4). Feedback-birth (§4.5), which also edits the newborn leg, is not drawn. The band below states where §3 finds the error and what §2 shows about how it fades.

Notation.

- Per neuron a: pre-activation mean μ_a, variance σ_a^2, $\alpha_a = \mu_a/\sigma_a$; κ3, κ4: third and fourth joint cumulants. Symbols with code names: Appendix B.00.
- Supply terms (O1, ACC, K31F, C2E, gk v3, tmd-lite, feedback-birth): Table 4.1. Other internal names: Appendix E. Evidence tags: §8.2.

Box 1. One layer of the shipped estimator (code order; lines of the linked `estimator.py`, sha8 dd17f7f4).

```
for ℓ = 0…15:                                                    6478
  μ ← Wᵀm;  time guards                                          6491–6513
  old sources join Q_c;  legs ← Wᵀ(q-scaled legs)                 6537–6831
  C_pre ← Wᵀ C W;  v, C_off                                      6832–6890
  D3, D21 from legs, slabs;  K31F, O1+ACC, GDK31 folds            6935, 7685–7963
  κ4 core G = diag(dG) + λ_ℓ C_off,  λ_ℓ rescaled per network    6946–6978
  Gaussian-ReLU Wick coefficients;  learned κ4 parts             7000–7043
  pair products → moments of y                                   7044–7178
  δ ← ridge heads, corrector (layers 6–15), last head (15)       7191–7206
  ℓ = 15: append m = E[y] + δ and stop                           7208–7211
  q ← gk v3 gate;  O1 store, ACC carry;  legs, D3, D21 × q       7214–7232
  newborn A = D_q C_off + C2E edit + feedback-birth              7247–7265
    κ4→κ3 feed with the tmd-lite metric                          7311–7338
  Mehler q3;  cumulants;  m ← E[y] + δ;  C;  newborn residual    7347–7519
  store dG, λ;  append row m                                     7603–7640
```

The code map is in Appendix E.

Sources: `research/arc-reference/`; `research/public-k3/`; `docs/writeup/evidence/official/lineage/LINEAGE-RECON-20261021.md`.

1.3 Derived, inherited, hand-set or learned

Table 1.1. What sets the output of #337350, grouped from the code bundle's `INVENTORY.md`.

group (size)	how set; shipped vs derived
ARC's propagation, 66 exact Wick coefficients; our constant-free terms (§§2, 4)	derived [derivation]; we pruned 10 more terms (−0.001 % MSE)
Inherited κ4 constant λ_ℓ, rescaled per network (33)	fitted by 504aldo; 2–5× physical; code: "part of this stack's accuracy lives in error cancellation"
Rank, age, width gates; alive and hub masks (≈ 20)	hand-set for cost [tuning networks]; three cuts cost 0.8–4.6 % official MSE each
Three ridge heads, a last-layer head (≈ 25,000); the corrector (18,663)	Monte Carlo labels; some sets unrecorded
Learned κ4 parts (≈ 11,000): A14 law, κ4-diagonal MLP, quartic-star mixing, K31 slot	Monte Carlo labels; several sets unrecorded
Gain tables: 26 band gains, 5 multipliers (31)	least-squares refits; neutral value 1
Hand-set strengths (7): feedback-birth, Mehler q3, O1, K31F, C2E, trace repair, lazy-join trace	scans on tuning or reused held-out networks: 0.38–0.41, 1.5, 1.5, 1.1, 0.65, 0.4, 2.0 (derived ½, rest 1)
Guard thresholds (≈ 60)	hand-set from official telemetry; off in §5

What this means.

- The skeleton is derived (§§2–4). On it sit about 55,000 fitted constants and seven hand-set strengths, four on §4 terms.
- Learned parts off: MSE +28 % (arm C). Strengths and gains at derived or neutral values: +1.3 % more (D over C). Inherited λ at a physical value alone: +71 % in the shipped build (§5).
- All switchable fits off (arm D′): +78–82 %, yet still 330× better than Monte Carlo at equal compute (shipped: 594×); 410 fitted constants (quartic-star mixing, K31 slot) have no switch.
- *Sampling is amortised offline, not absent.* Corrector versions saw up to 10,000 generated networks with 2^{24}-sample labels, about 2.6 million per-network Monte Carlo budgets, plus 741 public networks with 10^9-sample truth; the shipped round's set is unrecorded [unverified]. The "physical" λ is itself a Monte Carlo measurement on one network [simulation].

Sources: `docs/writeup/v9/APPENDIX-I-INVENTORY-v9.md` (= INVENTORY.md); `docs/writeup/EVIDENCE-APPENDIX-20261016.md`; `research/corpaci/research/astra_k4_overshoot.md`.

1.4 What predict does

Predict draws no random numbers and runs no input through the network; its one generator, from setup, is never read (line 6256). Range finders start from fixed slices of W or the previous basis (lines 6567, 6723, 7485). The inherited header (lines 25–67) describes a Monte Carlo fallback the file lacks (bundle README). Time guards can change a late predict's output (Appendix E).

Sources:
`experiments/claude_merge_20261014/package_c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065/package/estimator.py`; `docs/writeup/v9/BUNDLE-README-v9.md`.

2. The depth map fixes how errors fade

Any estimator makes a small error at every layer, and only the part that survives to the last layer reaches the score. For errors with no component along the mean, that part is a property of the random network, not of the estimator. It is the slope χ of the standard mean-field correlation map (Schoenholz et al., 2017), and here we use it as an error budget.

- With no fitted parameter it predicts the survival measured in our error ledger within 1–4 %.
- It predicts how the final error splits over the layers where it was made (Figure 2).
- It gives the inherited gate γ in closed form.
- It explains quantitatively why depth hurts.

2.1 One number per layer: the replica cosine

Let $h^{(\ell)} = W_\ell^\top y^{(\ell-1)}$ be the pre-activations of layer ℓ, with $y^{(-1)} = x \sim \mathcal{N}(0, I_n)$, $y^{(\ell)} = \mathrm{ReLU}(h^{(\ell)})$ and weights $\mathcal{N}(0, 2/n)$. For two independent inputs, the *replica cosine* is

$$c_\ell = \frac{\sum_a \mathbb{E}[h_a(x)h_a(x')]}{\sum_a \mathbb{E}[h_a(x)^2]} = \frac{\sum_a \mu_a^2}{\sum_a (\mu_a^2+\sigma_a^2)}, \qquad c_{\ell+1} = f(c_\ell) = \frac{\sqrt{1-c_\ell^2} + (\pi - \arccos c_\ell)\, c_\ell}{\pi}, \quad c_0 = 0$$

the arc-cosine map of degree 1 (Cho and Saul, 2009; Poole et al., 2016).

- On 12 fresh networks the measured c_ℓ is within 0.007 of the map on average, and within 0.05 on single networks [simulation].
- Across neurons, $\alpha_a = \mu_a/\sigma_a$ is approximately $\mathcal{N}(0, c_\ell/(1-c_\ell))$ [derivation].
- In deep layers the mean aligns with the top covariance directions (overlap with the top three 0.90 at layer 8, 0.96 at layer 15). α then becomes lighter-tailed than a Gaussian [simulation].
- §2.2 does not need α to be Gaussian; §2.4 does.

2.2 How much of an error survives

An error δ in the post-activation mean of layer $\ell - 1$ moves the next mean, to first order, by $J_\ell \delta = g_\ell \odot (W_\ell^\top \delta)$. Here $g_{\ell,a} = \Pr_x(h_a^{(\ell)} > 0)$ is exact, and the Gaussian closure gives $\Phi(\alpha_a)$; the two differ by about 0.1 % in what follows [simulation].

Claim. If δ does not depend on W_ℓ and has no component along the incoming mean, one layer keeps on average

$$\chi_\ell = \frac{\mathbb{E}\|J_\ell \delta\|^2}{\|\delta\|^2} = \frac{2}{n}\sum_a g_{\ell,a}^2 \;\longrightarrow\; 1 - \frac{\arccos c_\ell}{\pi} = f'(c_\ell)$$

of its energy.

Derivation.

- $(W_\ell^\top \delta)_a$ is Gaussian and independent of μ_a.
- Averaged over the Gaussian weight row, Sheppard's formula gives $\Pr(h_a(x) > 0, h_a(x') > 0) = \frac{1}{2} - \arccos(\rho)/(2\pi)$, and ρ concentrates at c_ℓ.
- Only the weights need be Gaussian. [derivation]

An error made in layer ℓ keeps $S(\ell) = \prod_{k=\ell+1}^{15} \chi_k$ of its energy at the output. Per network, S_{net} uses the network's own gates.

Table 2.1. Depth map and survival to the output (n = 1024).

- *Ledger:* end-layer over creation energy of each layer's error, 64 validation networks, 95 % intervals.
- *Fresh:* random directions through 12 new networks, mean ± standard error.

layer ℓ	c_ℓ	χ_ℓ	S(ℓ), law	S(ℓ), ledger	S(ℓ), fresh
0	0.000	0.500	0.027	—	0.029 ± 0.002
6	0.777	0.783	0.213	0.204 [0.197, 0.213]	0.224 ± 0.007
8	0.834	0.814	0.327	0.322 [0.310, 0.334]	0.340
10	0.872	0.837	0.473	0.479 [0.465, 0.494]	0.488 ± 0.012
12	0.897	0.854	0.654	0.651 [0.634, 0.667]	0.664
14	0.916	0.868	0.874	0.882 [0.870, 0.895]	0.875 ± 0.009

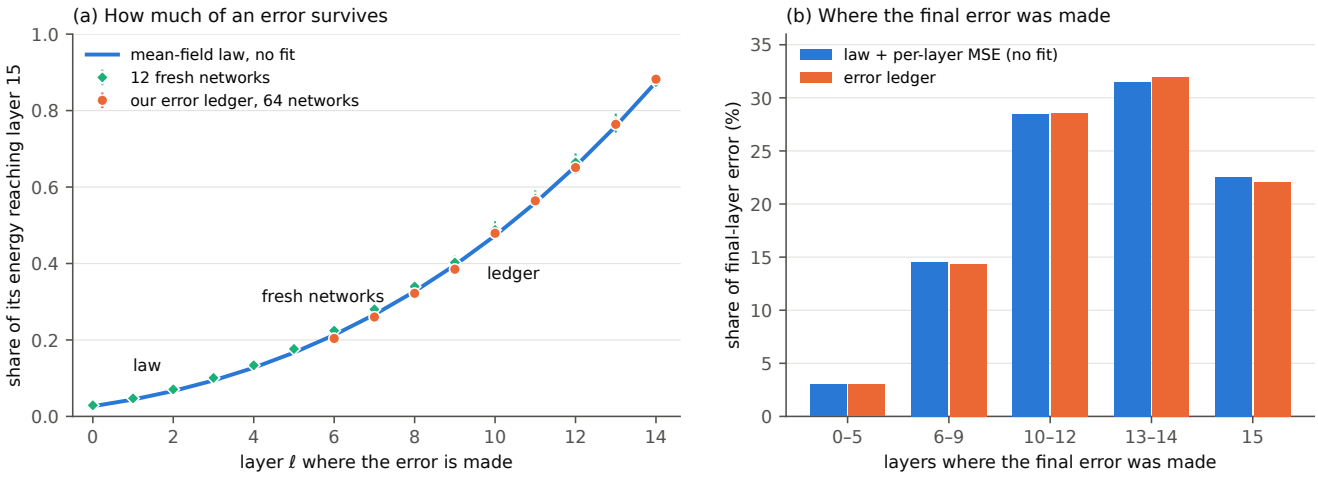


Figure 2. (a) Share of an error's energy that reaches layer 15, by the layer where the error is made: the mean-field law with no fitted parameter (line), our error ledger on 64 validation networks (circles, 95 % intervals) and 12 freshly generated networks (diamonds, ±1.96 s.e.). (b) Share of the shipped build's final-layer error by the layers where it was made: from the law and the per-layer MSE of 100 public networks (blue), and from the error ledger, computed from the actual error vectors on other networks, labels and build (orange).

Checks.

- *Ledger.* The law is inside the 95 % interval at layers 7–14 and at its edge at layer 6, with a law/measured ratio of 0.99–1.04 [64 validation networks].
  - The ledger propagates errors with the same linear map, so this tests the assumption about the errors' direction.
  - The variance, κ3 and κ4 cells survive alike.
- *Fresh networks.* They lie 5–9 % above the law at layers 0–7, and 0–3 % above it at layers 8–14. Single networks scatter by ± 11 %, which S_{net} tracks within ± 5 % [simulation].
- *Error budget by layer.* If each layer's new error adds in energy to the inherited one, then $\mathrm{MSE}_\ell = \chi_\ell \mathrm{MSE}_{\ell-1} + \mathrm{new}_\ell$ and $\mathrm{MSE}_{15} = \sum_\ell \mathrm{new}_\ell S(\ell)$.
  - Applied to the per-layer MSE of the shipped build on 100 public networks (§5, arm A), this gives the shares in Figure 2b: 3.0 / 14.6 / 28.5 / 31.5 / 22.5 % from layers 0–5 / 6–9 / 10–12 / 13–14 / 15.
  - The ledger, computed from the actual error vectors, gives 3.0 / 14.3 / 28.6 / 32.0 / 22.1 % [held-out, reused; derivation].
  - Two de-fitted engines (§5, arms D′ and F′) give nearly the same split. The split by layer is set by the network, and only the split by cell (§3.1) diagnoses the engine.

Sources: `docs/writeup/v9/calc/survival_law_v9.py`, `s2_sim_checks_v9.py`, `fig2_survival_v9.py` and their outputs; `experiments/claude_explore_20261014/ledger9a/budget_matrix_9A.json`; `experiments/claude_abl_20261022/results/abl_table_v1.json`.

2.3 The exception: the mean direction

An error along the incoming mean is a shared scale or bias error, and push and gate are then correlated. For Gaussian α and a constant σ_a, Stein's lemma gives $\chi_\ell^{\mathrm{mean}} = \chi_\ell + \frac{2c_\ell}{\pi}\sqrt{\frac{1-c_\ell}{1+c_\ell}}$: 0.96 at layer 6 and 0.99 at layer 15 [derivation].

- *Exact anchor.* A bias-free ReLU network is positively homogeneous, so an error that rescales the whole state never fades.
- *Simulation.* A mean-direction error made at layer 6 keeps 0.86 of its energy on average (12 networks; single networks 0.60–1.27), against 0.22 for a generic error [simulation].
- *Consequence.* Shared scale and bias errors are the dangerous kind in deep networks. The ledger's agreement with the generic law suggests our engine makes few of them; we did not measure that share.

2.4 The instantaneous coefficients and the gate γ in closed form

For Gaussian α, every neuron-averaged Gaussian–ReLU coefficient is a function of c (exact identities, checked by quadrature).

Table 2.2. Neuron averages for $\alpha \sim \mathcal{N}(0, c/(1-c))$, and how well they hold at layer 15 (measured over closed form, 6 networks) [simulation].

coefficient	enters	average	layer 15
$2\Phi(\alpha)^2$	error survival χ	$1 - \arccos(c)/\pi$	0.989
$\frac{2n}{n+4} r(\alpha)$	the gate γ	$\frac{n}{n+4}\left(1 - \sqrt{1-c^2}/\pi\right)$	0.988
$\varphi(\alpha) + \alpha\Phi(\alpha)$	ReLU mean per unit σ	$1/\sqrt{2\pi(1-c)}$	0.948
$\varphi(\alpha)$	curvature	$\sqrt{1-c}/\sqrt{2\pi}$	0.963
$(\alpha^2 - 1)\varphi(\alpha)$	κ4 injection	$-(1-c)^{3/2}/\sqrt{2\pi}$	0.597

- χ and γ hold within about 1 % at depth.
- The coefficients that weigh the tails of α drift: the κ4 injection is 40 % off at layer 15.
- The engine scales the carried correction ACC (§4.1) by $\gamma_\ell = \frac{2}{n+4} \sum_a r(\alpha_a)$, with $r = \Phi - \varphi^2 - \alpha\Phi\varphi = \partial\,\mathrm{Var}[\mathrm{ReLU}]/\partial\sigma^2$.
  - On 400 public networks the engine's γ, averaged over networks, is within 0.003 of the closed form at every layer. Single networks scatter by up to 0.027 [engine values].
- With θ = arccos c, γ and χ both equal 1 − θ/π to leading order.

Sources: `docs/writeup/v9/calc/survival_law_v9.py` (quadrature); `table22_shape_v9.py`; `docs/writeup/calc/gamma_meanfield_400.txt`.

2.5 What follows for depth

- **Forgetting is polynomial.** As c → 1, $1 - \chi_k \approx 3/(k + k_0)$ with k_0 between 8 and 11, so survival falls roughly as a cubic power of depth, not exponentially [derivation].
- **The layers that matter grow linearly with depth.** With equal error per layer, the output carries $\sum_\ell S(\ell)$ layers' worth: 3.8, 6.0, 10.3 and 18.6 at depths 8, 16, 32 and 64. In our engine the new error per layer also grows with depth (§3.1).
- **Why.** At layer 15 the spread of α is 3.46 (mean field) or 3.14 (engine). Three quarters of the neurons are then almost always on or almost always off, so errors pass with little loss, and carried cumulants persist (§3).
- **Where accuracy pays.** A fix at layer 6 is worth 0.21 of its local size at the output, and one at layer 14 is worth 0.87.
- **Assumptions.**
  - The weights are independent Gaussians and n is large.
  - The error is computed from earlier weights only.
  - The response is first order.
  - The generic law needs no mean component.

2.6 An independent check, and related forum work

Noise injection in ARC's reference chain. abhimanyu_singh (forum topic 18264) injected iid noise of size 10^{-3} into the D3 and D21 read-outs of a factored K = 3 chain at n = 256 and measured how much final-layer MSE it adds, by the layer where it was injected. Late layers were about 36 times more sensitive than layers 0–5. We repeated the experiment on ARC's reference implementation (`factored_nonlin_kprop_k3`, SIMPLE, n = 256, 16 layers, 8 networks, noise at one layer at a time). We split each layer's effect into the error made at that layer and the share of it that reaches layer 15.

- *Mean channel (D3 noise).* The share reaching the output matches S(ℓ) at layers 7–14 within 6 % (measured/law 0.94–1.04, mean 1.03). The law was not refitted; the intervention, the engine and the width differ from §2.2. At layers 1–6 the measured share is 1.2–1.6 times S(ℓ). Two effects account for it [simulation]:
  - *Finite width.* Noise added to the mean directly keeps more than S(ℓ) at early layers, by +54 %, +19 % and +12 % on average over layers 1–6 at n = 128, 256 and 512, and by at most 1–

2 % at layers 7–14 from n = 256. The excess shrinks roughly as 1/n; extrapolated to n = 1024 it is about +6 %, in line with the 5–9 % found on fresh n = 1024 networks in §2.2. It is not along the mean: that direction holds 0.1–0.5 % of the local error.

  - *A variance channel.* At early layers the same D3 noise also changes the variance, with 3–14 times the energy of the mean error at layers 1–6. A small share of a variance error reaches the output mean (10^{-3}–10^{-2}, measured by injecting into the variance directly). Adding the two channels, each with its own measured gain, predicts the final error at every layer within −22 % to +28 % (layers 1–14, mean ratio 1.00).

- *Covariance channel (D21 noise).* A covariance error first moves the mean one layer later and keeps feeding it at every later layer. Its share at the output is up to 2.6 times the error it first makes in the mean, so one survival factor does not describe it. This is the mechanism behind §3: the remaining error is made in the carried state, not in the mean [simulation].
- *The 36× profile.* We did not reproduce it. Our layer-window sensitivities are 1 / 11 / 42 / 83 / 113 for D3 alone and 1 / 8 / 19 / 19 / 8 for D3 and D21 together, against 1 / 9 / 24 / 30 / 36. That study adds noise per κ3 source of age ≥ 2, which ARC's single factored tensor does not have, so the mix of the two channels differs. The split above (local error × survival) applies to either.

Related forum work.

- abhimanyu_singh (18264) measured the layer profile and a bias/variance asymmetry: zero-mean read-out noise fades while truncation bias passes at gain ≈ 1. Our law gives the fading part in closed form.
- Artyom_Mazur (18249, Proposition 1) shows that a neuron's error at the next layer is a random projection of this layer's error through an independent weight column. That is the premise of the derivation in §2.2.
- il.miscusi (18248) and Artyom_Mazur (18249) remove a global scale mode inside their estimators (final-layer MSE ×0.380 in 18248). §2.3 predicts why this pays: an error along the scale direction does not fade.

Sources: `docs/writeup/v9/calc/inject36_v1/` (`inject36_v1.py`, `an36_v1.py`, outputs for 8 networks); `docs/writeup/v9/calc/inject36_v2/` (channel split and width scan: `inject36_v2.py`–`inject36_v4.py`, `an36_v3.py`, outputs).

Credit. χ is standard (Schoenholz et al., 2017). AI agents reviewing an earlier version of this write-up proposed using it as an error budget, together with the closed forms in c; we re-derived and checked both (§8).

References.

- Y. Cho and L. K. Saul, NeurIPS 2009.
- B. Poole et al., NeurIPS 2016.
- S. S. Schoenholz et al., ICLR 2017.
- W. F. Sheppard, Phil. Trans. R. Soc. A 192 (1899).

3. Where depth error lives

Most last-layer error enters the mean through the ledger's variance and κ3 cells (52 % and 31 %, §3.1). Behind them are two carried channels, the off-diagonal covariance C_off and the κ3 leg state; each layer refills both. Deep-layer truth oracles gain −29 % and −11 % of MSE respectively, overlapping by an unmeasured amount [tuning networks].

Notation. Per-source κ3 *legs* form the (2,1) slice **D21** ≈ κ(h_i,h_i,h_j); one layer's D21 is its *slot*, and sources older than four layers share the 352-column basis **Q_c**. **T** is the slice's true three-index part (a, b, c distinct), **X** = ½ Σ_cyc d_a q_b q_c κ4(h_a,h_a,h_b,h_c) the κ4→κ3 supply.

3.1 The last-layer ledger

Table 3.1. The ledger: exact shares (%) of last-layer error energy by cell and layer of creation, each piece carried to the output through diag(Φ)·W; 64 validation networks, 95 % bootstrap intervals [tuning networks].

cell	6-9	10-12	13-14	15	total
variance	7.3	15.4	17.4	11.3	51.5 [50.6, 52.4]

cell	6–9	10–12	13–14	15	total
κ3 + closure + heads	4.8	9.5	10.6	6.5	31.4 [30.5, 32.3]
κ4 after the corrector	2.2	3.7	3.9	4.2	14.1 [13.5, 14.7]
layers 0–5, unsplit	—	—	—	—	3.0 [2.6, 3.4]
all	14.3	28.6	32.0	22.1	100

- **Entry, not origin:** a covariance error counts where it enters a mean (§3.2).
- **Used networks:** these picked corrector versions and trained the heads; on 192 corrector-training and 532 other networks the shares move ≤ 0.3 points [tuning networks].
- **Calibrated, not shown exhausted:** the corrector's residual has zero signed overlap with its output, and doubling its data gave −0.059 % [−0.149, +0.030] (Appendix C.3) [held-out, reused].

Undoing §2's fading, the variance error made per layer grows from 4.3 % (layer 6) to 11.0–11.3 % (layers 14–15) of last-layer error [tuning networks].

Sources: `experiments/claude_explore_20261014/ledger9a/RESULT.md`; `docs/writeup/v9/calc/depth_error_checks_v1.py` (part A).

3.2 The covariance channel

Table 3.2. Truth oracles, corrector on [tuning networks]. C_off: 32 networks, 95 % bootstrap intervals. κ3: 16 networks, ± one standard error.

replaced by truth	MSE change	better
C_off, layer 14	−9.83 % [−10.51, −9.17]	32/32
C_off, layers 12–14	−24.25 % [−25.86, −22.56]	32/32
C_off, layers 11–14	−29.16 % [−30.89, −27.39]	32/32
κ3: three-index part in span(Q_c), layers 10–14	−11.03 ± 1.36 %	16/16

- **Three quarters is inherited:** with layer 13 exact, layer 14's own error is 23 % of its total at the output, and a layer fed exact input makes about 0.33 % relative C_off error against 0.64–0.71 % carried [tuning networks].
- **Not a corrector artefact:** with it off, single-layer oracles at layers 6–10 move ≤ 1.1 points [tuning networks].
- **What refills it:** the covariance update reads the D21 slot and the κ4 slots K31 and K22, so carried κ3 and κ4 errors feed C_off; hence the unmeasured overlap.

Sources: `experiments/claude_explore_20261014/c14re/RESULT.md`; `docs/WHAT-DIDNT-WORK-E.md`.

3.3 The κ3 channel: dynamics, not compression

The κ3 oracle's −11.03 % is not compression: the truth itself lies 95–96 % inside span(Q_c), so the error's 93–95 % there means nothing (Appendix G.1), and with exact old legs the representation share is 1 − 10.39/11.03 = 0.06 [−0.04, 0.15] [tuning networks]. **Confound:** the oracle ran with the corrector on, which reads D21 unmasked (truth in a layer-8 slot's empty block costs +9.41 % with it on, +1.40 % off), so part of the gain may be its response [tuning networks].

Sources: `docs/BIGBET-OLDTIER-TRI-20261020.md`; `docs/D21-E1-DEF-AUDIT-20261020.md`.

3.4 The leg state holds the κ3 error

The relative three-index error grows on average ×1.09 per layer in energy, to 0.140 at layer 14 (14 networks): a leaky store near its plateau, not compounding (Appendix G.3) [tuning networks]. Truth in one slot gains about a third of the five-layer oracle, but the next layer's error is back at 100–103 % of base; a layer-8 transplant that also filled the empty block lost +7.22 % [+5.13, +9.32], 1/14 better [tuning networks]. **A fix written into a slot does not enter the leg state; the unchanged legs regenerate the same error.**

Sources: `docs/BIGBET-OLDTIER-TRI-20261020.md`; `docs/writeup/v9/calc/depth_error_checks_v1.py` (part C).

3.5 The κ4→κ3 supply

In a one-step engine copy with true inputs (n = 1024), the true X removes 76–84 % of each layer's new κ3 error, noise-corrected (Table 3.3, Appendix G.3): three quarters to four fifths, on two networks and one step. The inherited feed form, without O1 or ACC, captures 24–34 % of X on network 624 [tuning networks]. **Toy width misleads:** a second-order template explains 82–98 % of the true κ4(a,a,b,c) at n = 32–64 [simulation] but 23–36 % at n = 1024 on network 624 [tuning networks].

Sources: `experiments/claude_ideas_20261014/onestep/README.md`; `experiments/claude_hist_20261015/toytri/RESULT.md`.

3.6 Why κ4 cannot be carried cheaply

Memory and conversion. On network 624 (slot layer 12) we replaced the pre-activation k = 1, 2, 4 layers back by a Gaussian of the same mean and covariance. What remains of X keeps $\beta = 0.04$, 0.10, 0.30 ($\beta = \langle \text{part}, X\rangle / \|X\|^2$ is a projection, $\rho = \|\text{part}\|^2/\|X\|^2 = 0.07$, 0.15, 0.33 an energy share): about 70 % is over four layers old. Of the newly made κ4, Gaussian birth projects only 14–15 %; the rest is mainly converted κ3 (higher orders not separated). Carried birth κ4 captures at most 2.6 % of X [tuning networks]. The memoryless feed of §1.2 cannot supply this.

Low rank. ARC proposes spending compute on high-variance covariance directions (forum topic 18078); the participation ratio falls from 0.50 n to 0.03–0.04 n by layer 15 (width 128–512) [simulation]. A rank-128 Tucker core in the top modes keeps 0.60–0.65 of X on network 624; at rank ≤ 16 ($\Delta F \approx 2$–7 %) it keeps 0.05–0.13 statically, below a break-even of 0.27–0.93 [tuning networks; cost and break-even: derivation]. Generating κ4 there costs $O(n^3)$ per source.

What §4 supplies. O1 and ACC (§4.1) are a cheap, partial supply of this converted κ4: O1 reads only young sources, and both enter through the (3,1) fold, not the feed X measures. Their share of X is unmeasured.

Sources: `experiments/claude_ideas_20261014/onestep/README.md`; `docs/K3SLICE-FEASIBILITY-20261020.md`; `docs/writeup/v9/calc/cov_participation_rank_v2.py`.

3.7 A sharp fitted optimum, and what is closed

Scaling the κ4 diagonal dG by 0.8 or 1.25 raises MSE by +408 % or +20,756 % (networks 544–548) [tuning networks], because dG feeds back through births and the inherited λ rule. The five-network optimum is within 0.3 % of the shipped value; −2 % or +2 % costs +6.5 % or +10.7 % [tuning networks].

A hypothesis, not tested. The inherited λ, fitted at 2–5× its physical value (§1.3), may stand in for the old κ4 of §3.6, consistent with the +71 % MSE at a physical λ in the shipped build (Table 5.3) [held-out, reused].

Reopen the κ3 channel when a representation carries the κ3 fragment, or its κ4 image, well below $O(n^3)$ per source and captures $X \geq 0.6$ in one step at n = 1024, measured before engine work; the covariance channel, when a state carries the cumulant slices feeding its update. For this engine structure, tuning networks closed per-source κ4 column transport, birth-only κ4 carriers, second-order templates (§3.5), slot-level corrections and cheap covariance fixes (Appendix C); both channels stay open.

Sources: `experiments/claude_loop_20261015/dgs/tab_dgs.py`; `experiments/claude_loop_20261015/fscan/tab_fscan.py`; linked `estimator.py` (lines 1450–1490, 1898–1904).

4. Derived supply terms

The inherited engine (§1.2) omits terms of ARC's cumulant expansion or carries them at the wrong strength. Seven supply terms restore some by editing objects it already forms, with no new n^3 product; Appendix B derives their strengths. §5's "six terms" exclude feedback-birth.

Table 4.1. The seven supply terms: derived (expansion) and shipped (linked estimator) strengths; an effect "at x" was measured at strength x.

term	adds	ARC counterpart	derived → shipped	effect [grade]
O1	κ4 core from κ3	counterterm	1 → 1.5	−0.869 % [−1.214, −0.516], 39/50 at 1 [official single change]

term	adds	ARC counterpart	derived → shipped	effect [grade]
ACC	O1, carried on	r = 1 trace	1 → 1	−0.519 % [−0.854, −0.175], 34/50 [official single change]
K31F	(3,1) slice in D21	κ4 (3,1) term	1 → 1.1	−1.66 % [−2.05, −1.26], 69/89 at 0.8, layers 6–14 [held-out, reused]
C2E	doubled-edge birth	doubled light edge	1 → 0.65	−1.56 % [−1.90, −1.22], 75/89 at 0.5 [held-out, reused]
gk v3	dressed leg gate	vertex dressing	no scalar	−0.366 % [−0.518, −0.208], 194/320 [held-out, used once]
tmd-lite	neuron-wise feed metric	transported metric	no scalar	MSE −0.246 % [−0.371, −0.127], 106/164 [held-out, reused]
feedback-birth	full-rank D21 feedback	one-heavy-leg paths	½ → 0.38–0.41	−9.93 % [−10.75, −9.09], 50/50 at ¼ [official single change]

4.1 O1 and ACC

The engine's κ4 core, a fitted multiple of the covariance, misses ARC's counterterm: the inherited κ3 contracted with one vector. O1 adds it for three matrix–vector products per source: contracting the legs with e first (B.4), it folds

$$X_F = c_A\, D_q\, T_e\, D_q, \qquad (T_e)_{bc} = \sum_{a \notin \{b,c\}} e_a\, \kappa_3(a,b,c), \qquad e = 2m(1-q), \qquad c_A = \frac{6}{n+4}$$

into young sources' slabs at layers 2–14 through the next layer's legs $A' = W_{\ell+1}^{\top} D_q A$ and P':

$$\Delta L_A = \rho \odot (P' D_{c_a} + A' D_{c_b}), \qquad \Delta L_P = \rho \odot (A' D_{c_a} + P' D_{c_d}), \qquad \rho = -\tau\, c_A\, \frac{\mu}{3v}.$$

ACC carries it on: one gate passes the core as $\gamma_\ell D_q G D_q$, so one scalar per layer suffices (B.5, §2.4),

$$\gamma_\ell = \frac{2}{n+4} \sum_a r(\alpha_a), \qquad r = q - md, \qquad \mathrm{acc}_\ell = g\,\gamma_\ell\, \mathrm{acc}_{\ell-1} + \mathrm{fresh}_\ell.$$

The shipped τ = 1.5 (B.9) beats the derived 1 by 2.4 % on the de-fitted engine (Table 5.3) [held-out, reused]. Both official single changes ran at the derived strengths. The contract-first fold and the γ carry are ours.

Sources: `docs/writeup/SECTION-O1-KAPPA3-SUPPLY-20261017.md`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

4.2 K31F and C2E

K31F. D21 ignores the (3,1) slice κ4(h_i,h_i,h_i,h_j), which enters the next covariance. Polarizing the *quartic star source* $F_a = \sum_r \sum_k c_{k,r} A_{ar}^{4-k} P_{ar}^{k}$, the engine's fit of each source's κ4 diagonal, gives that slice as rows on D21's existing right operands, $K_{31} = U_A A^{\top} + U_P P^{\top}$:

$$U_A = c_0 A^3 + \tfrac{3}{4} c_1 A^2 P + \tfrac{1}{2} c_2 A P^2 + \tfrac{1}{4} c_3 P^3, \qquad U_P = \tfrac{1}{4} c_1 A^3 + \tfrac{1}{2} c_2 A^2 P + \tfrac{3}{4} c_3 A P^2 + c_4 P^3.$$

They join young sources' slabs at layers 3–14 with weight $\tau_{31}\, g_s\, t/3d$. The rows are exact only for the fitted quartic, the weight only for the slice's covariance term (B.6; strength: B.9). Credit: the folding clue of the public #332868 (marius_binner); polarization rows from an LLM (ChatGPT) consulted by the participant, checked by us.

C2E. The newborn source misses ARC's doubled light edge, two parallel covariance edges on one side of a path. An elementwise edit of the newborn leg adds it at layers 1–14:

$$\Delta A_{ij} = \tfrac{1}{2}\,\tau_{\text{birth}}\,d_i\,C^2_{\text{off},ij}\,\frac{t_j}{d_j}, \qquad i \neq j.$$

τ_birth is derived 1 (B.3), shipped 0.65 (B.9); on the de-fitted engine the two agree within 0.3 % (Table 5.3) [held-out, reused]. ARC's term; the public #333054 (marius_binner) folds "N = 3 path trees into newborn tree legs"; ours is the port.

Sources: `docs/writeup/SECTION-K31-FOLD-DRAFT-20261016.md`; `experiments/claude_pair_20261014/fresh/res_writeup_20261017/verdict_arms.txt`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

4.3 gk v3: the slope of the mean

The engine gates the κ3 legs, D21 and the newborn leg by the Gaussian slope q = Φ(α), though its mean uses each neuron's κ3 and κ4; to first order the gate is that mean's slope (B.7), about ten elementwise operations per layer:

$$g = \partial_\mu m = q + \tfrac{\kappa_3}{6}W_4 + \tfrac{\kappa_4}{24}W_5 + \tfrac{\kappa_3^2}{72}W_7 + \tfrac{\kappa_3\kappa_4}{144}W_8.$$

Its κ4 input includes a learned correction. The effect (Table 4.1) has Holm p = 2.3e-6, at unchanged compute [held-out, used once]. ARC dresses only the covariance vertex; dressing the leg gate is ours.

Sources: `experiments/team4/compute/gk_gate_20261015/fragment.py`; `experiments/claude_pair_20261014/fgrun_g02/report_v1/RESULT.json`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

4.4 tmd-lite

The κ4→κ3 feed weights each newborn row by 2, the expected squared norm of a neuron's input weights, not by the actual norm the engine already computes. In the feed term $\frac{1}{12}M_{aa}\,d_a\,q_b G_{bc} q_c$ (B.8) tmd-lite sets

$$M_{aa} = 2\omega_a/\bar{\omega}_\ell, \qquad \omega_a = (W_\ell^\top W_\ell)_{aa},$$

a 4 % correction [simulation]. Credit: il.miscusi's "tmd-lite", with josepha_mayo's paired evidence #337047 → #337046; ours are the port and the link to ARC's transported metric.

Sources: `docs/writeup/SECTION-3_4-TMD-20261021.md`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

4.5 Feedback-birth

Each newborn κ3 source should hold D21's cross terms with the covariance. The inherited engine carried them in thin legs of rank 16, cut to 6 in #334775; feedback-birth puts them into the newborn leg at full rank, with no new product:

$$A_{\text{new}} = D_q C_{\text{off}} + s_\ell \left(D_d D_{21} + D_q D_{21}^\top D_{t/d}\right).$$

The derived ½ reproduces ARC's terms exactly at first order; the thin legs keep one of ARC's two orientations, hence ¼ (B.2) [derivation]. The official step #334775 → #334780 shipped that ¼ at full rank; the derived ½ was never an official single change. The shipped 0.383–0.412 comes from scans and band gains (B.9). On the de-fitted engine ½ beats ¼ by 17 % of MSE, 100/100 networks; in the shipped build, switching feedback-birth off alone raises the MSE 3.7-fold (Table 5.3) [held-out, reused]. Credit: public designs #333139 (shiv_m) and #333656 (mk1981; 0.5 on two heavy channels, whether its normalisation matches ours is [unverified]); ours are the derivation, code, measurements and the derived ½.

Sources: `results/intel-public-descriptions-20261007/descs.json`; `experiments/codexb_compute/incumbent-feedback-birth-v1/algebra-proposal.txt`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

4.6 What paid, what did not, and what is ours

Strengths. O1 and K31F ship above their derived strength, C2E and feedback-birth below, the rest at it, each chosen on a partly fitted chain (B.9).

Not every derived term paid (Appendix C):

- full K22 birth port: MSE +1.21 %, compute +17.9 % [tuning networks];

- exact supplies: host-κ4 K31 slice +0.7 % to +5.0 %; one-layer κ3→κ4→κ3 leg supply −0.68 % at compute +175 % [tuning networks];
- ARC ports: capped (2,1,1) terms with the counterterm, +0.56 % and −0.17 % balanced, +334 % unbalanced; 3+2 tree in the leg transport, +76 %, 0/32 [tuning networks];
- gate one order up: MSE +0.27 %, score +0.45 % [held-out, reused];
- feedback-birth ×2.3 on the three-index channel, falsified by a one-step test (Appendix D) [tuning networks].

What is ours. New: O1's fold, ACC's γ carry, gk v3's leg-gate dressing. Our derivations of public designs: feedback-birth (½), K31F (fold weight). Ports: C2E, tmd-lite.

GDK31, the (2,2) part of the next layer's K31 slice folded in through the youngest source, is not one of the seven. It ships at 1, which the code bundle's inventory calls derived; we have not re-derived it [unverified]. Bundle #336766 → #336836: −0.48 % [−0.73, −0.24], 35/50 [official bundle].

Sources: Appendices C and D; `docs/writeup/evidence/official/LEDGER-STEPS-20261021.md` (step 63); `docs/writeup/calc/ledger_pairs_v1.py`.

5. Where the score gain came from

The official ledger (Table 5.1). Full-budget Monte Carlo has MSE σ^2/N = 1.132e-6 (N = 65,374 samples), 84× ours; at our compute 590×, against 224× at the start [official bundle]. By kind tag (from build recipes), the accuracy half splits [official bundle]:

- steps with any fitted part (35): 87.9 %, or 93.3 % counting three steps with untagged tuned constants, not all from the fit; fit-only steps (10): 31.5 %;
- our mechanisms in steps with no fitted part (7): 28.7 %, strictly 23.3 %; feedback-birth alone 15.9 %; O1 and ACC 3.1 %;
- three lossy cuts: −16.9 %, buying 28.8 % of the compute gain.

These are sequence-order shares: O1 and ACC entered at steps 56–57, after 26 fitting steps had absorbed part of the same error, and feedback-birth at step 14.

Table 5.1. Official records on the 50 public networks, each step against its nearest scored ancestor [rows 1–2: official bundle; rows 3–4: official single change].

what	records	result
start → end	#332675 → #337350	score 5.0495e-9 → 1.9177e-9, −62.0 %, 50/50; MSE 2.188e-8 → 1.346e-8; compute fraction C 0.2308 → 0.1425
accuracy vs compute	69 steps	ln(score ratio) = −0.968 = −0.486 (MSE) − 0.483 (C) + 0.0002 (cross term): accuracy 50.2 %, compute 49.9 %
feedback-birth at ¼	#334775 → #334780	−9.93 % [−10.75, −9.09], 50/50; MSE −7.42 %
O1; ACC	#336621 → #336682; #336682 → #336706	−0.87 % [−1.21, −0.52], 39/50; −0.52 % [−0.85, −0.18], 34/50

Sources: `docs/writeup/v9/calc/ledger_split_v9.py`; `docs/writeup/evidence/official/LEDGER-STEPS-20261021.md`; `.venv/lib/python3.10/site-packages/whestbench/budget.py` (lines 179–278).

What the fitted parts carry: an ablation of #337350. We switched groups of fitted parts off on 100 public mini-split networks (10^9-sample truth) and 64 others never used for any decision (full-split shards 9–11 and 60; some heads' training records are incomplete). Each network's nested arms ran on one machine, time guards off, so values are independent of machine speed. All 2,212 predictions completed; no guard fired; every switch held.

Table 5.2. Nested ablation: final-layer MSE over A's (95 % bootstrap interval) and advantage over Monte Carlo at equal compute (× MC) [mini: held-out, reused, but the ablation selects nothing; 64: held-out, used once].

arm	switched off (cumulative)	ratio, mini	ratio, 64 once-used	× MC, mini/64
A	nothing (as shipped, 312 GF)	1	1	594/586
B	learned heads and corrector	1.148 [1.137, 1.159]	1.150 [1.137, 1.163]	521/513
C	B + learned κ4 parts	1.278 [1.261, 1.295]	1.278 [1.261, 1.295]	470/464
D	C + strengths at derived values, gains at 1	1.294 [1.276, 1.313]	1.296 [1.277, 1.315]	464/457
D′	D + λ at a physical value	1.818 [1.783, 1.854]	1.783 [1.740, 1.827]	330/332
E	B + O1, ACC, K31F, C2E, gk v3, tmd-lite	1.257 [1.243, 1.271]	1.261 [1.243, 1.279]	478/471
E′	D′ + the same six terms	1.947 [1.910, 1.987]	—	310/—
F	start #332675 (507 GF)	1.682 [1.651, 1.713]	1.594 [1.558, 1.630]	217/226
F′	#332675's port of ARC's k_max = 3 chain, κ4 regeneration off (524 GF)	2.995 [2.930, 3.063]	2.847 [2.764, 2.932]	118/123

Every arm was worse than A on every network of both sets.

Table 5.3. Single switches and strength checks, 100 mini networks [held-out, reused]: final-layer MSE over the base arm's (95 % bootstrap interval).

arm	change	base	ratio	better than base
λ only	physical λ	A	1.709 [1.674, 1.746]	0/100
feedback-birth only	feedback-birth off	A	3.737 [3.638, 3.839]	0/100
six terms only	E's six terms off	A	1.098 [1.090, 1.106]	1/100
λ = 0	no off-diagonal κ4 core	D	1.739 [1.707, 1.773]	0/100
feedback-birth ¼	inherited ¼, not derived ½	D′	1.167 [1.158, 1.176]	0/100
C2E 0.65	shipped, not derived 1	D′	1.003 [1.002, 1.005]	36/100
O1 1.5	shipped, not derived 1	D′	0.976 [0.974, 0.977]	100/100
start, λ physical	#332675, physical λ, no per-network rule	F	1.529 [1.499, 1.560]	0/100

These arms ran on other machines than their bases; on the 60 networks whose runs shared a CPU family, ratios agree within 0.025.

Sources: `experiments/claude_abl_20261022/` (runners, receipts, tables); `docs/writeup/v9/ABLATION-SPEC-v9.md`; `docs/writeup/v9/ABLATION-RUN-NOTES-v9.md`.

Reading.

- *Rule fixed before the run:* D′ far worse than A means the score rests on fitted parts; D′ raises the MSE by 78–82 %, so it does, substantially.
- *Feedback-birth is the largest mechanism:* off alone, the MSE nearly quadruples; on D′ its derived ½ beats its inherited ¼ by 17 % (§4.5).
- *The six other terms are worth about 10 %:* 9.8 % as shipped, 9.5 % over B, 7.1 % over D′, above the sequence-order 3.1 %.
- *Against Monte Carlo at equal compute:* ARC's reference chain as ported (F′) 118–123×, the start with a physical λ 142×, D′ 330×, shipped 594×.
- *Like for like* (physical λ, no learned parts), our changes cut the MSE by 29 % at 61 % of the compute; D′ has 8 % higher MSE than the start as shipped.

- "*330×" is against plain sampling at equal FLOPs,* not stronger methods, and not fit-free: D′ keeps 410 fitted constants with no clean switch, the pruned Wick tables and the compression gates.
- Mean propagation alone, at almost no compute, beats Monte Carlo about 130× at its own compute, without the score's 0.1 compute floor (starter-kit figures).

Sources: `docs/concepts/problem-setup.md`.

6. What this means

Each implication names its evidence, with the tags of §8.2.

6.1 For ARC's question: why estimators break down with depth

1. **Depth hurts through slow forgetting.** An error made at layer ℓ keeps $S(\ell)$ of its energy at the output, and S falls only polynomially (§2.5).
   - With equal error per layer, the output carries 3.8, 6.0, 10.3 and 18.6 layers' worth of error at depths 8, 16, 32 and 64.
   - So, going from depth 16 to 64 at a fixed output error, each layer's new error must shrink about 3×. In our engine it grows with depth instead (§3.1). [derivation; 64 validation networks]
2. **Shared errors are the dangerous ones.** An error along the mean, a shared scale or bias, keeps 0.96–0.99 of its energy per layer (§2.3). A mechanistic estimator for deep networks should get the mean direction exactly right first. [derivation; simulation]
3. **What is missing is memory.** Three quarters to four fifths of each layer's new κ3 error comes from κ4 that is old state converted from κ3 (§3.6).
   - A closure that maps this layer's means and covariance to the next one cannot supply it, however good it is.
   - The estimator has to carry more state through depth, or a compressed history. [two networks, one step]
4. **Derivation can replace fitting, but has not yet.** With every switchable fitted part off, the engine is still 330× better than Monte Carlo at equal compute; the shipped build is 594× (Table 5.2).
   - Where we could test a derived strength against a fitted one, the derived one won: feedback-birth at ½ against ¼, by 17 %.
   - The remaining factor of about 1.8 is structure that the fits capture and our derivations do not. It is a measured target for theory. [public networks, ablation]

6.2 For people building estimators

- **Weight accuracy work by survival.** A fix at layer 6 is worth 0.21 of its local size at the output, and one at layer 14 is worth 0.87 (§2.5). The law takes a few lines of code (`checks/depth-law/survival_law_v9.py` in the repository). [derivation]
- **Fix the carried state, not the readout.** Truth oracles gain 29 % in the covariance and 11 % in the κ3 legs. A corrected value written into one layer's slot does not enter the state (§3). [tuning networks]
- **Fold derived corrections into products you already compute, and report the strength you ship.** Feedback-birth's official −9.93 % was measured at strength ¼ (§4). [official single change]
- **Do not save compute by discarding directions.** A corrector recovered about 71 % of a magnitude loss but about 0 % of a direction loss (Appendix C). [tuning networks]
- **Keep a fresh network set for decisions.** Effects measured on fresh networks carried over to official results at 1.07×, and effects measured on networks reused for selection at 0.29× (Appendix G.2). [official bundle]
- **Test in the grader's mode before trusting a local number.** Two of the tool issues below pass every in-process test and fail only under the grader's client–server mode or its memory limit (§6.3).

6.3 For the benchmark and its tools

We checked each item below on the current releases (flopscope 0.12.3, whestbench 0.16.2), both in-process and through flopscope's own client–server pair. The scripts are in the repository's `feedback/` folder, and the details are in Appendix H. None of T1–T4 appears among the 113 open and closed issues of flopscope's tracker as of 22 October.

Each item gives its kind, what happens, and its effect on a participant.

- **T1** (bug). `sum`, `mean` and `max` with a tuple `axis` raise `TypeError` in client–server mode only, because the client sends tuples as lists. *Effect:* Code that passes every local test fails in grading. Behind a fallback guard, it silently ships the fallback.
- **T2** (bug; patch verified). `einsum` with three or more operands, or with a repeated operand, allocates a float64 buffer the size of each operand for its cost analysis (four sites). *Effect:* Twice the bytes of a float32 operand. Fails under the grader's 8 GiB limit where NumPy passes.
- **T3** (NumPy difference). In client–server mode, an `out=` call returns a new wrapper, not `out` itself. *Effect:* Caches keyed on the returned object miss: about 1 % extra requests for us.
- **T4** (documentation). The wheel ships `weights.csv`, whose "Active Weight" column differs from billing for 96 of 461 operations (`take` 0 against 4). *Effect:* Wrong cost estimates if the file is read as the price list.
- **S1** (scoring design). A failed network scores zero predictions at multiplier 1.0, about 0.01 added to a 2e-9 score. In grading, one time-out also stopped the run. *Effect:* One failure loses the submission, so we built timing guards, which make some outputs timing-dependent.
- **S2** (scoring design). The 5 s setup window includes process start and imports, with five workers starting at once, while `whest validate` times `setup()` alone. One overrun fails the whole submission. *Effect:* One setup took 2.8 s alone on our machine, 4.5 s on four grading workers and over 5 s on the fifth (#337287). Two of our ~17 submissions near the limit failed outright.
- **S3** (scoring design). Compute counts one-for-one with accuracy. *Effect:* Half of our 62 % gain is compute engineering that says nothing about estimation.

Suggestions.

- *For S1:* cap a failed network's score, for instance at a fixed multiple of the submission's median, or score a trimmed mean. A guard would then be optional, and outputs could stay deterministic.
- *For S2:* make `whest validate` time the same window as grading (spawn, imports and setup, ideally with five workers), and report each worker's setup margin in the evaluation record.
- *For S3:* also rank accuracy at a fixed compute, so that mechanistic gains show apart from compute engineering.

6.4 Open problems

1. **A cheap κ4 memory.** A generator for each source's κ4 well below $O(n^3)$ would supply most of the κ3 error that is still missing (§3.6).
2. **Error sizes from first principles.** §2 fixes how errors fade. A law for how large each layer's new error is would complete the budget.
3. **Less compute without discarding directions.** A leading submission matches our final-layer MSE at 0.75× our compute (§7). Every cut we tried that dropped directions lost accuracy.
4. **The fitted factor of 1.8.** Which structure do the learned corrections capture (§6.1, item 4)?
5. **Beyond random Gaussian ReLU networks.** The survival law assumes independent Gaussian weights. Trained networks and other activations are untested.

7. Limitations

- **The ablation (Tables 5.2–5.3)** ran on public networks, not the official ones, in-process, guards off; single switches ran on other machines than their bases (same-family subsets agree within 0.025). Its "physical" λ is a one-network Monte Carlo measurement, its most de-fitted arm keeps 410 fitted constants, and its nested shares are sequence-order shares.
- **Most local evidence is optimistic.** Effects of reused held-out sets transferred to new networks at about 0.29×, of once-used sets at 0.76–0.87× (Appendix G.2); only official records are clean.

- **§3 rests on few networks:** two networks and one step for the κ4→κ3 share, one for memory and low-rank capture. The κ3 oracle ran with the corrector on; the overlap of the two oracle gains is unmeasured.
- **§2 needs Gaussian weights** and fixes how errors fade, not their size; trained networks are untested.
- **Two predictions recorded before their tests failed:** a hashed explanation of where the κ4-row fold's gain leaks missed its joint criteria, and a recommended ×2.3 feedback-birth strength failed a one-step test [tuning networks].
- **Time guards make some outputs timing-dependent** (§1.4). Officially one network shows a lever-sized compute rise, with no limit flag. #337451 lost 1.78 %, most likely (unconfirmed) through a sticky lateness rule.
- **The prize score comes from a fresh run** on new networks; one time-limit failure voids a submission.
- **The compute gap stayed open.** A leading submission (#337292) matches our final-layer MSE (×0.985) at ×0.75 our compute, 234 against 312 GF [official, another team's public record]; our direction-discarding cuts all lost accuracy.
- **The tool feedback (§6.3, Appendix H)** was reproduced on one machine, with flopscope's client–server pair built from source rather than the hardened grading image. The einsum patch was tested on flopscope's own tests and six expressions, not inside a full submission. S1–S3 rest on our own submissions.
- **Records are incomplete.** The learned parts' training-set manifest is missing, some fitted parts lack a recorded training set, and one §3 input array came from a scratch folder (its sha256 matches).

Sources: `experiments/claude_ideas_20261014/` (leak-law pre-registration, one-step test); `experiments/claude_setupbox_20261020/post337451/`; `experiments/claude_explore_20261014/intel_E/data/evals/evaluation_337292.json`.

8. LLM use, attribution, licences and reproduction

8.1 How the work was done

The participant (megan_park).

- The participant set the research direction and proposed most of the ideas that were tested.
- The participant also set the goals, the priorities, and the resource and evaluation rules; decided which directions to pursue or stop; and decided every official submission.
- From about 4.5e-9 (7 October, #334675) onward, the agents repeatedly concluded that further gains were out of reach. The participant kept proposing new directions, and the official score fell to 1.92e-9 by 20 October (#337350; Appendix A).

The AI agents implemented the ideas, derived the formulas, and ran and checked the experiments: Claude models (Anthropic) via Claude Code, in fixed-role sessions, which also made the authorised official submissions; OpenAI Codex, for evidence documentation, cost analysis and static review, structural research and run-time validation; and ChatGPT (web interface), consulted by the participant for the K31 polarization rows, checked in B.6. Earlier sessions did not record model versions; the 22 October revision used Claude Opus 5.5.

"We" in this write-up means the participant together with the AI agents. Unless stated otherwise, the agents carried out the derivations, code and numerical checks; the ideas behind them came mostly from the participant.

This document. AI agents drafted it from the participant's results and decisions; the participant read and approved it. Three of five AI reviewers, prompted to take ARC researchers' perspectives, proposed §2's error budget and closed forms; we re-derived and checked both. The agents found the tool issues of §6.3 during the competition and re-checked them on 22 October on the current releases; they also wrote the einsum patch and its checks.

Most ideas came from the participant. That does not mean a person re-derived every formula or reread every line of code. The agents did those and checked them, and the tags of §8.2 say how each claim was checked. The participant takes final responsibility for this write-up, the submission and this disclosure.

8.2 Evidence tags

tag	basis
[official single change], [official bundle]	AIcrowd records (Appendix A); a bundle changed several things at once
[held-out, used once], [held-out, reused]	paired runs on networks outside all training and tuning; "reused" sets informed several decisions, so are optimistic
[tuning networks]	networks that chose structures or constants: evidence for choices, not values
[simulation]	freshly generated random networks
[derivation]	text or Appendix B; numerically checked where a script is named
[unverified]	reported, not re-checked from a file

Agreement between agents is not counted as replication.

8.3 Attribution and licences

- ARC's cumulant propagation, counterterm identity and C2edge birth family.
- 504aldo's V29 engine and fitted constants; corpaci's Phase 2 package #332675 (§1.2).
- PLinOpt's rank-48 4×4×4 programs (Dumas, Grenet, Pernet, Sedoglavic; CeCILL-B); public V34 alive-neuron masks (acco-cyber, koushik_rudra; MIT).
- Public ideas we derived and built: the folding clue of #332868, feedback-birth from #333139 and #333656, the C2E edit after #333054, and il.miscusi's tmd-lite with josepha_mayo's paired evidence #337047 → #337046 (§4).
- Standard theory (§2): Cho and Saul (2009), Poole et al. (2016), Schoenholz et al. (2017), Sheppard (1899).

Licences. The submitted tar holds only `manifest.json` and `estimator.py`, whose header carries the 504aldo and V34 MIT notices. The published code bundle (https://github.com/Meganpark980320/arc-whitebox-p2-337350) adds `LICENSE-arc`, `LICENSE-public-k3`, `LICENSE-corpaci` and `LICENSE-acco-cyber` with `NOTICE-acco-cyber.md` (all MIT), PLinOpt's CeCILL-B text with `AUTHORS.md`, and a `LICENSE` naming "AIcrowd (starter kit)" and "meganpark (this work)"; the estimator still shares imports, class and method signatures and two one-line statements with the starter kit.

8.4 Reproduction

- *Artefact:* #337350, estimator sha256 dd17f7f4…, tar f44bb724… (Appendix A). Appendix E maps §§2–4 to estimator lines; the header's Monte Carlo fallback never runs (§1.4).
- *Checks:* the §§2–4 scripts (`docs/writeup/v9/calc/`) run in minutes on a laptop (Appendix F).
- *Not in the repository:* Monte Carlo truth arrays, recorded chain states and training labels (Google Drive, on request; Appendix F.7), and a complete manifest of the learned parts' training sets (missing).

Appendix A. The linked submission #337350

A.1 Official record

Table A.1. AIcrowd record of #337350, submitted 20 October 14:36 UTC (whestbench 0.16.1, flopscope 0.12.3) [official bundle].

quantity	value
score, 50 public networks	1.918e-9 (leaderboard 1.9177e-9)
final-layer MSE	1.346e-8
compute per network	313 GF of 2^41 ≈ 2,199 GF (14.2 %)
networks	100/100 completed, no limit flags
timing (limit)	setup per worker 3.92–3.98 s (5 s); predict median 103.4 s, max 108.6 s (120 s); residual client time median 0.349 s, max 0.366 s (0.4 s)
full-budget Monte Carlo MSE	1.132e-6: 84× ours; 590× at equal compute, since it scales as 1/compute [derivation]

Sources: `experiments/claude_liaison_local_20261017/official_eval/eval_337350_C_fin4_cf4g065_completed.json`; `.venv/lib/python3.10/site-packages/whestbench/budget.py` (lines 179–183).

A.2 Package and last step

The tar (sha256 f44bb724…) holds `manifest.json` and `estimator.py` (sha256 dd17f7f4…, 21,067 lines; mapped in Appendix E). The last step, #337303 → #337350, retrained the corrector, lowered a fitted compression-loss gain from 0.5 to 0.4, raised the C2E strength from 0.5 to 0.65 and cut setup and request work without changing predictions: −0.637 % [−0.883, −0.388], 38/50 [official bundle].

Sources: `submit-queue/submitted/f44bb724_c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065.tar.gz`; `docs/writeup/evidence/official/LEDGER-STEPS-20261021.md` (step 69).

A.3 How to rerun

With whestbench 0.16.1, `whest validate-package <tar>` and `whest validate --estimator estimator.py` check the package; `whest run --estimator estimator.py --dataset <baked dataset>` evaluates it against a flopscope server, about 100 s per network. Local timings differ from official ones; we did not rerun it here.

Appendix B. Derivations behind the supply terms (§4)

[part X] is the part, A to J (not an appendix), of `docs/writeup/v9/calc/supply_terms_checks_v9.py` that checks an identity on small random tensors or by exact quadrature. Line numbers refer to the linked `estimator.py` (sha8 dd17f7f4) and to `research/arc-reference/src/mlp_kprop/`.

B.00 Symbols

Table B.0. Symbols and their code names.

symbol	meaning	code
ℓ, n; W_ℓ; f, y	layer, width; weights, $h = W_\ell^\top y$; ReLU, output	`li`, `n`; `w32` (`W = w32.T`)
$\mu_a, v_a = \sigma_a^2, \alpha_a$; m_a; e	pre-activation mean, variance, ratio; post-activation mean; $2m(1-q)$	`mu`, `var`, `alpha`; `pk1v`; `e0`, `e_b`
W_k; q, d, t; g; r, γ_ℓ, c_A	Wick coefficients, $q = W_1$, $d = W_2$, $t = W_3$ (B.0); gk v3 gate; $q - md$, ACC's carry factor, $6/(n+4)$	`W_all`; `w1`, `w2`, `w3`, `r31` = t/d; `_gk_transport_gate`; `gam`
C_{off}; D_v, Sym	off-diagonal covariance; diagonal matrix of v; symmetrisation	`C_off`
D3, D21; L_A, L_P; source; birth; young	κ3(a,a,a), κ3(a,a,b); their left slabs; a layer's κ3 (B.0); its new source; age < 4	`D3`, `D21`; `lap`; `A_st`, `P_st`, `s_list`; `a_b`; `ka`

symbol	meaning	code
dG; G, λ_ℓ; F_a, g_s; U_A, U_P; $\omega_a, \bar{\omega}$	κ4 diagonal; inherited κ4 core diag(dG) + λ_ℓ C_off; quartic star source, gain; its K31 rows; $(W_\ell^\top W_\ell)_{aa}$, expectation 2	`dG`; `lam_prev`; `Om`, `_CI_G['star']`; `_ci_k31f_fold`; `_cf_wwrs`, `METRIC_C`
τ, g, τ_{31}, τ_birth, s_ℓ	strengths of O1, ACC, K31F, C2E, feedback-birth	`CI_O1P_TAU`, `CI_O1P_ACC`, `CI_K31F_TAU`, `CI_C2E_TAU`, `0.35*_CI_G['fbb']`

Sources: the linked estimator; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.0 Conventions

- $W_k = \mathbb{E}\, f^{(k)}(\mu + \sigma Z)$. For ReLU, $W_0 = \sigma(\varphi + \alpha\Phi)$, $q \equiv W_1 = \Phi(\alpha)$ and $W_k = (-1)^k\, \mathrm{He}_{k-2}(\alpha)\, \varphi(\alpha)\, \sigma^{1-k}$ for $k \geq 2$, so $d \equiv W_2 = \varphi/\sigma$, $t \equiv W_3 = -\alpha\varphi/\sigma^2$ and $t/d = -\mu/v$; also $\mathbb{E}[(f^2)'] = 2m$, $\mathbb{E}[(f^2)''] = 2q$, $\mathbb{E}[(f^2)'''] = 2d$.
- **Source**, the κ3 born at one layer: legs A_s, P_s, birth vectors y_s (that layer's d), η_s, ϵ_s (its e) and a rank-8 residual $Z_s L_s^\top$, Z transported:

$$\kappa_3^{(s)} = \mathrm{Sym}(3A_s,\, P_s,\, A_s D_{y_s}) + \mathrm{Sym}(M_s, P_s, P_s), \qquad M_s = P_s D_{\eta_s} + 3A_s D_{\epsilon_s} + Z_s L_s^\top,$$

with $\mathrm{Sym}(X,Y,Z)_{ijk}$ the symmetrised $\sum_r X_{ir} Y_{jr} Z_{kr}$; each layer sets $A \leftarrow W_\ell^\top D_q A$ (estimator 7223–7228 apply D_q; the product with $W_\ell^\top$ is taken in the next layer's transport, 6774–6787).

- **Slabs.** $D_{21,ij} = \kappa_3(h_i, h_i, h_j)$, $i \neq j$, is

$$D_{21} = \sum_s \left(L_{A,s} A_s^\top + L_{P,s} P_s^\top \right) + \tfrac{1}{3} \sum_s (P_s \circ P_s) L_s Z_s^\top,$$

$$L_A = 2A \circ P \circ y + P \circ P \circ \epsilon, \qquad L_P = A \circ A \circ y + \tfrac{1}{3} P \circ P \circ \eta + \tfrac{2}{3} M \circ P$$

Supply terms edit only these left slabs or the newborn leg [part J: D_{21}, D_3 match $\kappa_3^{(s)}$ to 1e-14].

- **Birth.** $A = D_q C_{\mathrm{off}}$, $P = I$ (estimator 7247).

Sources: `research/arc-reference/src/mlp_kprop/tensor_utils.py`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.1 Covariance to all orders, cross-cumulants to first order

Keeping C to all orders and the cross-cumulants $\kappa(a^p b^q \cdots)\, s_a^p s_b^q \cdots / (p!q!\cdots)$ of order ≥ 3 to first order, the characteristic function is the product of the marginals times $\exp\left(-\sum_{a<b} C_{ab} s_a s_b\right)$ times $1 + \sum \kappa(\cdot)\, (is_a)^p (is_b)^q \cdots / (p!q!\cdots)$; integrating by parts gives, for any F_a,

$$\mathbb{E}\Big[\prod_a F_a(z_a)\Big] = \prod_a \mathbb{E}_a[F_a] + \sum_{\text{blocks}} \frac{\kappa(a^{p_a}, b^{p_b}, \ldots)}{p_a!\, p_b! \cdots} \prod_a \mathbb{E}_a\big[F_a^{(p_a)}\big] + \ldots,$$

with $\mathbb{E}_a$ under the full marginal; k parallel edges carry $C_{ab}^k/k!$, and joint cumulants keep connected diagrams. With the cumulant-of-products identity (Leonov–Shiryaev) this gives [derivation; part B, softplus quadrature: ratios 0.9999–1.0001, disconnected control 3e-16]:

- T1b, centre edge: $\kappa_3(y_a, y_b, y_c) \ni \frac{1}{2}\, t_a\, q_b\, q_c\, \kappa_3(a,a,b)\, C_{ac}$.
- T1c, single-index edge: $\frac{1}{2}\, d_a\, d_b\, q_c\, \kappa_3(a,a,b)\, C_{bc}$.
- C2edge, doubled light edge: $\frac{1}{2}\, d_a\, t_b\, q_c\, C_{ab}^2\, C_{bc}$; one edge gives the birth term $q_a\, d_b\, q_c\, C_{ab} C_{bc}$.
- Vertex dressing: $\mathrm{Cov}(y_a, y_b) \ni \frac{1}{6}\, W_4(a)\, q_b\, \kappa_3(a,a,a)\, C_{ab}$.

Sources: `docs/writeup/v9/calc/supply_terms_checks_v9.py`; `docs/writeup/v9/calc/supply_terms_checks_v9.out.txt`.

B.2 Feedback-birth: the derived coefficient is ½

504aldo's feedback legs (estimator 1530–1543) give the newborn source

$$X = 3A + \tfrac{3}{2} D_d D_{21}, \qquad Y = A D_d + \tfrac{1}{2} D_q D_{21}^\top D_t, \qquad B_1 = \mathrm{Sym}(X, I, Y),$$

with D_{21} projected to rank 16, later 6. With $F = A + \frac{1}{2} D_d D_{21}$ and $G = A + \frac{1}{2} D_q D_{21}^\top D_{t/d}$, $X = 3F$ and $Y = G D_d$. Completing the square,

$$\mathrm{Sym}(X, I, Y) = 3\,\mathrm{Sym}(B, I, B D_d) - 3\,\mathrm{Sym}(E, I, E D_d), \qquad B = A + \tfrac{1}{4}\left[D_d D_{21} + D_q D_{21}^\top D_{t/d}\right], \quad E =$$

a newborn source with leg B minus a signed source quadratic in D_{21}. #334780 kept B at full rank and dropped the E-pair, so its coefficient, and the one its official step measured (§4.5), was ¼: the thin-leg strength at full rank [derivation; part C: identity to 1e-17]. For $A \to A + \delta$, $\delta = c\,(D_d D_{21} + D_q D_{21}^\top D_{t/d})$, the δ-linear birth term $6\,\mathrm{Sym}(\delta, I, A D_d)$ equals $2c$ times T1b + T1c (B.1) on all-distinct entries, summed over orderings: $c = \frac{1}{2}$ [part C: ½ matches to 5e-16, ¼ gives half]. ARC lists each heavy leg in both orientations (`factor_k3.py`:391–393, 401–407); the inherited legs keep one. Terms quadratic in D_{21} are dropped; the companion leg must come from B, $S_{\mathrm{sep}} = D_\epsilon B^\top$, or $3(B - A) D_\epsilon$ enters uncompensated (estimator 7481).

Sources: `experiments/codexb_compute/incumbent-feedback-birth-v1/algebra-proposal.txt`; `research/arc-reference/src/mlp_kprop/factor_k3.py`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.3 C2E: the derived strength is 1

The edit $\delta = \frac{1}{2}\,\tau\, D_d\,(C_{\mathrm{off}} \circ C_{\mathrm{off}})\, D_{t/d}$ has a birth-linear part equal on all-distinct entries to τ times C2edge (B.1), as does ARC's doubled-light-edge leg (`factor_k3.py`:398–399). So τ = 1 [derivation; part C: matches to 5e-16].

Sources: `research/arc-reference/src/mlp_kprop/factor_k3.py`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.4 O1: ARC's counterterm, contracted into the legs

By the cumulant-of-products identity,

$$\kappa_4(y_a, y_a, y_b, y_c) = \kappa_3(y_a^2, y_b, y_c) - 2m_a\,\kappa_3(y_a, y_b, y_c) - 2\kappa(y_a, y_b)\kappa(y_a, y_c).$$

To first order in the inherited κ3 the first two terms give $e_a\, q_b\, q_c\, \kappa_3(a, b, c)$, $e = \mathbb{E}[(f^2)'] - 2mq = 2m(1 - q)$. Projected onto ARC's r = 1 core by 6/(n + 4) (B.5), this is ARC's counterterm (`factor_k3.py`:470–521), its $-2c_A$ and c_A merged into e:

$$X_F = c_A\, D_q\, T_e\, D_q, \qquad (T_e)_{bc} = \sum_{a \notin \{b,c\}} e_a\, \kappa_3(a, b, c), \qquad c_A = \frac{6}{n + 4}.$$

Contracting first, with $\alpha = A^\top e$, $\pi = P^\top e$, $\zeta = Z^\top e$ for one source (B.0) [part J: 1.5e-14],

$$T_e = A\, D_{c_b} A^\top + A\, D_{c_a} P^\top + P\, D_{c_a} A^\top + P\, D_{c_d} P^\top + \tfrac{2}{3}\,\mathrm{Sym}(Z L^\top D_\pi P^\top),$$

$$c_a = y \circ \alpha + \epsilon \circ \pi, \qquad c_b = y \circ \pi, \qquad c_d = \eta \circ \pi + \epsilon \circ \alpha + \tfrac{1}{3} L\zeta$$

(the thin last term is off). The next core $W_{\ell+1}^\top X_F W_{\ell+1}$ is a sum of products of the transported legs $A' = W_{\ell+1}^\top D_q A$, $P' = W_{\ell+1}^\top D_q P$. Under ARC's average metric ($\bar\omega = 2$, B.5) its (3,1) slice is $K_{31,ij} = (\bar\omega/2)(W_{\ell+1}^\top X_F W_{\ell+1})_{ij}$, and adding $(t_i/3d_i) K_{31,ij}$ to D_{21} reproduces its covariance term (B.6). So, with derived τ = 1 [part J: 3e-15],

$$\Delta L_A = \rho \odot \left(P' D_{c_a} + A' D_{c_b}\right), \qquad \Delta L_P = \rho \odot \left(A' D_{c_a} + P' D_{c_d}\right), \qquad \rho_i = \tau\, c_A\, \frac{t_i}{3d_i} = -\tau\, c_A\, \frac{\mu_i}{3v_i}$$

(estimator 17561–17583). Dropped: $a \in \{b, c\}$ and the diagonal of T_e, smaller by $n^{-1/2}$ [derivation; not measured]; the counterterm's newborn half (a separate switch, off); later layers, restored by ACC. Cost: about 0.51 GF per predict [official single change], against 56 GF for the dense fold [derivation; part H].

Sources: `research/arc-reference/src/mlp_kprop/factor_k3.py`; `experiments/claude_flops_20261014/o1prod/DESIGN.md`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.5 ACC: ARC's r = 1 trace under its average metric

ARC stores κ4 as $\mathrm{Sym}(G \otimes M)$, core G and metric M (`harmonic.py`:21–22). For a zero-diagonal core and $M = \bar{\omega} I$,

$$\big[\mathrm{Sym}(G \otimes \bar{\omega} I)\big]_{aakl} = \frac{\bar{\omega}}{6}\, G_{kl} \quad (a \notin \{k,l\}), \qquad \sum_a \big[\mathrm{Sym}(G \otimes \bar{\omega} I)\big]_{aakl} = \bar{\omega}\, \frac{n+4}{6}\, G_{kl} \quad (k \neq l),$$

so 6/(n + 4) times the doubled-index trace returns the core for $\bar{\omega} = 1$ [part E]. By B.4 and B.1, $\kappa_4(a,a,k,l)$ enters $\kappa_4(y_a, y_a, y_k, y_l)$ as

$$\tfrac{1}{2}\big(\mathbb{E}[(f^2)''] - 2m\,\mathbb{E}[f'']\big)_a\, q_k q_l = \big(\mathbb{E}[f'^2] + \mathbb{E}[f f''] - m\,d\big)_a\, q_k q_l,$$

for ReLU $r(\alpha_a)\, q_k q_l$ with $r = q - md = \Phi - \varphi^2 - \alpha\Phi\varphi$ [part B: quadrature to 1e-6]. Tracing without the terms $a \in \{k,l\}$ and the row sums of G, one order of n smaller [derivation],

$$G'_{kl} = \frac{6}{n+4} \sum_{a \notin \{k,l\}} r_a\, q_k\, \frac{\bar{\omega}}{6}\, G_{kl}\, q_l = \gamma_\ell\, q_k G_{kl} q_l, \qquad \gamma_\ell = \frac{\bar{\omega}}{n+4} \sum_a r(\alpha_a) = \frac{2}{n+4} \sum_a r(\alpha_a)$$

[part E: 1e-16], with $\bar{\omega} = 2$, ARC's average metric for He-scaled layers (`kprop_harmonic.py`:511, 578–579); the exact one-hop metric is untested. The D_q factors travel with the legs, so each layer multiplies O1's stored vectors by γ_ℓ; the q in γ and e is the gk v3 slope (B.7).

Sources: `research/arc-reference/src/mlp_kprop/kprop_harmonic.py`; `docs/review-20261017b/writeup-judge.md`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.6 K31F: polarization rows and the fold weight

The engine fits each source's κ4-diagonal contribution as a quartic in its legs (estimator 9736–9782),

$$F_a^{(s)} = \sum_r \sum_{k=0}^{4} c_{k,r}^{(s)}\, A_{ar}^{4-k} P_{ar}^{k},$$

the diagonal of $K_4^{(s)} = \sum_r \sum_k c_{k,r}\, \mathrm{Sym}\big(a_r^{\otimes(4-k)} \otimes p_r^{\otimes k}\big)$ for leg columns a_r, p_r. Polarization, $K_k(x^{k-m}, y^m) = \frac{(k-m)!}{k!} D_y^m F_k(x)$ for $F_k(x) = K_k(x, \ldots, x)$, gives with k = 4, m = 1 $K_{31,ij} = K_4(e_i, e_i, e_i, e_j) = \frac{1}{4}\partial_{x_j} F_4$ at $x = e_i$:

$$K_{31} = U_A A^\top + U_P P^\top, \quad U_A = c_0 A^3 + \tfrac{3}{4} c_1 A^2 P + \tfrac{1}{2} c_2 A P^2 + \tfrac{1}{4} c_3 P^3, \quad U_P = \tfrac{1}{4} c_1 A^3 + \tfrac{1}{2} c_2 A^2 P + \tfrac{3}{4} c$$

with elementwise powers [part D: matches the explicit four-tensor to 6e-14]. By B.1, K31 enters the post-activation covariance as $\frac{1}{6} t_i q_j K_{31,ij}$ and D_{21} as $\frac{1}{2} d_i q_j D_{21,ij}$ (estimator 1669), so adding $(t_i/3d_i) K_{31,ij} = -(\mu_i/3v_i) K_{31,ij}$ to D_{21} reproduces the covariance term: derived $\tau_{31} = 1$. In the forward (2,1) term, where D_{21} carries $\frac{1}{2}\mathbb{E}[(f^2)''] q_j = q_i q_j$ and K31 $\frac{1}{6}\mathbb{E}[(f^2)'''] q_j = d_i q_j / 3$, the weight is off by $q_i t_i / d_i^2 = -\alpha_i \Phi(\alpha_i)/\varphi(\alpha_i)$, unbounded for usually-on neurons (about −36 at α = 2); births and the next state also read the folded slice [derivation]. A (2,2) slice needs quadratic right factors (closed, Appendix C).

Sources: `docs/writeup/SECTION-K31-FOLD-DRAFT-20261016.md`; `docs/EXPERT-S2-20261016.md`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.7 gk v3: the gate as the slope of the mean

By B.1, to first order in the cross-cumulants, C included,

$$\mathrm{Cov}\big(f(h_a), f(h_b)\big) = \sum_{p,q \geq 1} \frac{\kappa(a^p, b^q)}{p!\, q!}\, \mathbb{E}_a[f^{(p)}]\, \mathbb{E}_b[f^{(q)}].$$

Its (1,1) term $C_{ab}\, \mathbb{E}_a[f']\, \mathbb{E}_b[f']$ uses each neuron's own marginal, with $\mathbb{E}_a[f'(h_a)] = \partial_\mu \mathbb{E}_a[f(h_a)]$ at fixed shape, as does every vertex of the κ3 transport. First-order transport is thus gated by the slope of the marginal mean at fixed cumulants, q at zeroth order; the (2,1) term is carried by D_{21},

and dressing it is second order [derivation]. The engine's mean is its table's five-term Edgeworth expansion (estimator 1663–1667),

$$m = W_0 + \tfrac{\kappa_3}{6} W_3 + \tfrac{\kappa_4}{24} W_4 + \tfrac{\kappa_3^2}{72} W_6 + \tfrac{\kappa_3 \kappa_4}{144} W_7,$$

with $\kappa_3 = D_{3,a}$ and κ4 the corrected κ4 diagonal. As $\partial_\mu W_k = W_{k+1}$,

$$g = \partial_\mu m = q + \tfrac{\kappa_3}{6} W_4 + \tfrac{\kappa_4}{24} W_5 + \tfrac{\kappa_3^2}{72} W_7 + \tfrac{\kappa_3 \kappa_4}{144} W_8, \qquad W_8 = \mathrm{He}_6(\alpha)\, \varphi(\alpha)\, \sigma^{-7}$$

[part B: dressing coefficient 1/6 to 6e-5]. In ARC's terms this is vertex dressing (`partitions.py`:469–477), inside k_max = 3 on covariance edges (block cost 3) but not on the κ3 legs (cost 4, or 6 for κ3², κ3κ4).

The truncation. Under ARC's power counting for zero-mean weights, per-entry size $n^{-c/2}$ with $c = |v| - 1 - [\text{all entries even}]$ (it roughly holds, with coefficients growing in depth [unverified]), the table keeps every n^{-1} term of the mean but only $\kappa_3^2 W_6/72$ and $\kappa_3 \kappa_4 W_7/144$ of the five n^{-2} terms (not $\kappa_4^2 W_8/1152$, $\kappa_5 W_5/120$, $\kappa_6 W_6/720$): an inconsistent truncation the gate inherits [derivation]. Zero-mean weights cancel the naive $n^{-3/2}$ order of $\kappa_3 \kappa_4$, κ5 and κ_3^3 (slopes in n −0.98, −1.00, −0.50 for $\sum_j W_{ja}^3$, $\sum_j W_{ja}^4$, $\sum_j |W_{ja}|^3$) [simulation, part F].

Sources: `research/arc-reference/src/mlp_kprop/kprop_harmonic.py`; `research/arc-reference/src/mlp_kprop/partitions.py`; `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.8 tmd-lite: the metric on the doubled index

For $\kappa_4 = \mathrm{Sym}(G \otimes M)$ with a diagonal metric and $a \notin \{b, c\}$, one pairing survives, $\kappa_4(a,a,b,c) = \frac{1}{6} M_{aa} G_{bc}$ (B.5), so the feed term of B.1 is $\frac{1}{2} d_a q_b q_c\, \kappa_4(a,a,b,c) = \frac{1}{12} M_{aa}\, d_a\, q_b G_{bc} q_c$. The inherited feed sets $M_{aa} = \bar{\omega} = 2$. tmd-lite sets $M_{aa} = 2\omega_a/\bar{\omega}_\ell$, with $\bar{\omega}_\ell$ the layer mean of $\omega_a = (W_\ell^\top W_\ell)_{aa}$, the diagonal of the exact one-hop metric ($\mathbb{E}\,\omega_a = 2$, $\mathrm{sd} = \sqrt{8/n} = 0.088$ under He initialisation) [part G]; ARC's k_max = 1 linear step uses exactly these norms (`kprop_harmonic.py`:296–301). Off-diagonal metric entries, of order $n^{-1/2}$, are dropped.

Sources: `research/arc-reference/src/mlp_kprop/kprop_harmonic.py`; `docs/writeup/SECTION-R-20261020_v3.md` (§3); `docs/writeup/v9/calc/supply_terms_checks_v9.py`.

B.9 How the shipped strengths were chosen

- **O1, τ = 1.5:** alone −0.416 % [−0.563, −0.267], 116/164 [held-out, reused]; officially, with a guard change, −0.109 % [−0.418, +0.184], 23/50 [official bundle].
- **K31F, τ_{31} = 1.1:** scanned over 0.6–1.4 on tuning and corrector-training networks [tuning networks]; 0.8 → 1.1 came with exact compute cuts, −0.63 % [−0.89, −0.37], 38/50 [official bundle].
- **C2E, τ_birth = 0.65:** against the first 0.5, −0.291 % [−0.402, −0.176], 111/164, but failing our tail rule, under which the worst network may lose at most the mean change plus three expected chance scatters [held-out, reused].
- **Feedback-birth, s_ℓ = 0.383–0.412**, 0.35 times a fitted band gain [part A]. A ×1.0–1.8 scan of the ¼ of #334780 on 32 tuning networks peaked at ×1.4, the literal 0.35 [tuning networks]; it entered officially with exact cost cuts, #334806 → #334933: −3.92 % [−4.57, −3.25], 47/50 [official bundle]. On this partly fitted chain the optimum lies below ½: the newborn leg also feeds the fitted quartic κ4 source, K31F and O1.

Sources: `docs/HANDOFF-20261008.md` (lines 73, 145); `experiments/claude_explore_20261014/all164_E/agg_e21d_final.txt`; `docs/writeup/calc/ledger_pairs_v1.py`.

Appendix C. Scoped negative results

Each row closes one candidate on one parent build and network set, not its family. Changes are final-layer MSE, negative better, unless marked; "est." is modelled. [tuning networks] marks tuning or training sets: they close candidates, never accept them.

C.1 κ3/κ4 supply and transport

ID	candidate	result	networks
N26	Exact host-κ4 slice for the fitted K31 slot	+0.7 to +5.0 %	32 [tuning networks]
N02	Exact one-layer κ3→κ4→κ3 leg supply	−0.68 % at compute +175 %	16 [tuning networks]
N31; N36; J	Full K22 birth port	+1.21 % [+0.61, +1.82] at compute +17.9 %	32 [tuning networks]
N27	ARC's capped (2,1,1) terms with the counterterm	+0.56 / −0.17 % balanced, +334 % unbalanced	32 [tuning networks]
K.2	Rescaled or refitted K31 slots; 3+2 tree, dense or absorbed into transport	+3.3 / +10.6 / +31 %; oracle −3.19 %, port compute +60 to +200 % (est.); absorbed +76 % (0/32)	16, 12; 32 [tuning networks]
OLDTIER-TRI	Three-index oracle with exact old legs	representation share 0.06 [−0.04, 0.15] (§3.3)	16 [tuning networks]
transplant	Truth in one layer's slot	−2.72 % (layer 8), −3.5 to −3.8 % (layers 10, 12); next layer back at 100–103 % of base (§3.4)	14 [tuning networks]
X supply	One engine step with true inputs	X is 77 % of the new three-index error; engine forms capture ≤ 34 % of X, per-birth κ4 carry ≤ 3 % (§§3.5–3.6)	544, 624 [tuning networks]

Sources: `docs/writeup/WRITEUP-v6.8-20261018.md`; `docs/BIGBET-OLDTIER-TRI-20261020.md`; `experiments/claude_ideas_20261014/onestep/README.md`.

C.2 Covariance and the transport gate

ID	candidate	result	networks
K.3; cov dictionary	Cheap covariance fixes; 28-knob dictionary	top-1 % pairs −0.04 %; pair-feature fits +0.15 / +0.07 %; dictionary −0.003 % cross-validated	21; 32 [tuning networks]
GK2	Gate one order further	+0.274 % (score +0.448 %)	164 [held-out, reused]

Sources: `docs/writeup/WRITEUP-v6.8-20261018.md`; `docs/writeup/SECTION-I-20261020_v2.md`.

C.3 Learned correctors

ID	candidate	result	networks
K.1	Double the corrector's data	−0.059 % [−0.149, +0.030], 92/164	164 [held-out, reused]
S-track	Rank cut plus trained twin cells	+6.89 % (rank 288), +17.62 % (rank 256); cut-specific cells recover 0.6 % and 3.2 %: cells rescale; the cut loses directions	96–127 [tuning networks]

Sources: `docs/writeup/WRITEUP-v6.8-20261018.md`; `experiments/claude_strack_20261020/RESULT.md`.

C.4 Rank and state cuts

ID	candidate	result	networks
G7	Rank-288 subspace with the most last-layer influence	+12.35 % against +5.75 % for the static basis; linear influence misses directions middle layers use	8 [tuning networks]

Sources: `docs/writeup/OUTLINE-NUMBERS-20261021.md`.

The full list, including K4FR (per-source κ4 column transport, §3.7), ships with the code bundle: `docs/writeup/APPENDIX-C-NEGATIVE-20261021_v3.md`.

Appendix D. Pre-registered tests

We registered predictions before reading some results. Two failed their tests:

- An explanation of a variance "leak" across layers was rejected on its own joint criteria.
- A theory-based recommendation to raise feedback-birth to 2.3× failed a one-step test with true inputs.

A locked specification for a held-out comparison decided one choice even though its registered constant was pessimistic. The full records ship with the code bundle.

Appendix E. Code map, guards and cost engineering

E.1 Code map

File: `experiments/claude_merge_20261014/package_c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065/package/estimator.py` (sha8 dd17f7f4, 21,067 lines). Later sections re-bind earlier functions.

piece	lines
entry points	`Estimator` 7971 → `_V25Core` 6235; outermost setup 19412 (of 16 bindings), predict 21066 (of 9) → … → `_predict_core` 6345 (Box 1)
O1; ACC	store 17457 (call 7221), fold 17561, τ 17440; γ 17765–17768
K31F; GDK31	7859 → 18230 → 18189 (GDK31) → 17633 (O1) → 17222 (K31F); τ 17193
C2E; feedback-birth	17351 (call 7254), τ 17335; 7255–7265
gk v3; tmd-lite	20082 (call 7216); 6968–6972, 7317–7324
heads; corrector; last-layer head	8022 (fits 8049, 8052, 8085); 14947 (call 12512); 17163
learned κ4 parts	A14 6110; quartic star 15538; κ4-diagonal MLP 18846; K31 slot 7063–7095
gains; λ	1892–1899; table 1772, per-network rule 6952–6961
guards; safety net	E.3; 14044 → 14009 (Gaussian covariance, else mean field, else zeros)

Switches. Unset, the 246 environment variables give the submitted build. The ablation (§5) uses `CI_O1P_TAU`, `CI_O1P_ACC`, `CI_K31F_TAU`, `CI_C2E_TAU`, `ARC_GK_GATE`, `CI_TMD_LITE`, `CI_K4C_TAU`, `CI_TR_GAIN`, `CF_TRLZ_G`, `V25_BETA`, `CI_GK` and, after import, `NO_CORR=True` (heads, corrector), `est.core._law_arrays=None` (A14), `_CF_RG_ON=False` and `_CF_DL_TRIGGER_S=_CF_DL_TRIGGER2_S=1e9` (guards). `V17_NO_CORR=1` makes setup raise (line 9871).

Dead code. Unused: the header's matched-Monte-Carlo blend fallback and comparison arm (lines 25–67), the blend's constants (371–381), the comparison chain (385–1436) and its switch `_V25_STACK` (1467).

Sources: the estimator; `docs/writeup/v9/ABLATION-SPEC-v9.md`; `docs/writeup/v9/ABLATION-RUN-NOTES-v9.md`.

E.2 Internal names

name	meaning
`c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065`	#337350's leaderboard description: our build name, chaining its ancestors' tags
old tier, Q_c; lazy join	rank-352 basis of sources over four layers old; lazy joins (from layer 11) reuse it
slot; K31, K22	one layer's stored D21; κ4 slices (i,i,i,j), (i,i,j,j)
K31 slot; GDK31	fitted K31 stand-in; §4.6
quartic star; A14 law	§4.2; learned κ4-diagonal correction
Mehler q3	$C^3/6$ term of the covariance's Mehler series
trace repair; lazy-join trace	re-inject κ3 trace lost to compression (layers 10–14), to lazy joins (13–14)

name	meaning
alive, hub masks	public V34 masks; skip low-weight columns of young D21 products

Sources: `docs/writeup/evidence/official/lineage/LINEAGE-RECON-20261021.md`; `docs/writeup/v9/APPENDIX-I-INVENTORY-v9.md`.

E.3 Time guards and #337451

- Near the 0.4 s residual limit, a residual guard switches on *levers*, lighter paths, from layer 3 (20869–20962); a deadline guard drops one fast-product step at a projected 111 s (13786–13797).
- Three levers, like the designed light paths of a setup's first two predicts (13739–13809), only reorder floating-point operations; the *head lever* (20953) skips later learned mean corrections, +18 to +32 % MSE on that network [unverified].
- The *sticky rule* (20873–20876, 21039–21057) carries a late verdict into the worker's next predict.
- Officially only network 4 shows a lever-sized compute rise (+1.9 %), with no limit flag; whether the head lever fired is unrecorded [official bundle].

#337451 scored 1.952e-9, 1.78 % worse than #337350 [official bundle]. Our hypothesis (its reproduction was stopped): the sticky rule kept re-arming, as the levers save too little real time to disarm it, so one normal-speed worker stayed on lighter paths for 18 networks and carried most of the loss.

Sources: the estimator; `experiments/claude_setupbox_20261020/post337451/GUARD-MARGIN-REPORT-20261021_v3.md`; `experiments/claude_pair_20261014/fresh/official/evaluation_337451.json`.

E.4 Cost engineering

- Compute fell 38.3 % from #332675 to #337350 [official bundle].
- Largest cuts: alive-neuron masks −6.5 % (MSE +4.6 %), thin feedback-leg ranks −4.9 % (MSE +3.0 %), exact root-product folding −1.8 % [official single change]; shared-buffer Strassen–Winograd routing −5.8 %, lazy joins at layers 13–14 −2.1 %, PLinOpt's programs −1.2 % [official bundle].
- Other pieces batch requests, reuse buffers or move work into setup; predictions change only by rounding.
- flopscope bills an m×k by k×n matmul 2mkn − mn FLOPs; fast programs split it into smaller ones plus billed additions.

Sources: `docs/writeup/evidence/official/LEDGER-STEPS-20261021.md`; `.venv/lib/python3.10/site-packages/flopscope/_flops.py`.

Appendix F. Evidence

Paths are repository-relative; a sha8 is a sha256's first eight hex digits.

F.1 Official records

record	file	sha8
#337350, linked	`experiments/claude_liaison_local_20261017/official_eval/eval_337350_C_fin4_cf4g065_completed.json`	c99749c9
#337350 package	`submit-queue/submitted/f44bb724_c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065.tar.gz`	f44bb724
#337350 estimator	`experiments/claude_merge_20261014/package_c2eko10S2H8D3ahF10CR3GsHV4RxDnoT78cf4g065/package/estimator.py`	dd17f7f4
#337303	`experiments/claude_liaison_local_20261017/official_eval/eval_337303_fp3_resubmit_completed.json`	8967700a

record	file	sha8
#337264	docs/writeup/evidence/eval_337264.json	ad896f42
#337287, failed setup	docs/writeup/evidence/eval_337287.json	e4af1483
#337451	experiments/claude_pair_20261014/fresh/official/evaluation_337451.json	02e5e61f
45 more, incl. start #332675; public list #332600–#337453	docs/writeup/evidence/official/	584938e1 (SHA256SUMS.txt)

F.2 §2

Sources: docs/writeup/v9/calc/ (survival scripts and outputs); experiments/claude_explore_20261014/ledger9a/budget_matrix_9A.json; docs/writeup/calc/gamma_meanfield_check.py (features: F.7).

F.3 §3

Sources: docs/writeup/v9/calc/depth_error_checks_v1.py; docs/BIGBET-OLDTIER-TRI-20261020.md; experiments/claude_ideas_20261014/onestep/README.md.

F.4 §4

Sources: docs/writeup/v9/calc/supply_terms_checks_v9.py (Appendix B identities); experiments/claude_pair_20261014/fresh/res_writeup_20261017/verdict_arms.txt (89 networks); experiments/claude_pair_20261014/fgrun_g02/report_v1/RESULT.json (gk v3, 320 networks).

F.5 §5

Sources: docs/writeup/v9/calc/ledger_split_v9.py; docs/writeup/evidence/official/LEDGER-STEPS-20261021.md; experiments/claude_abl_20261022/ (ablation).

F.6 How to check

```
sha256sum <file> | cut -c1-8                # F.1
(cd docs/writeup/evidence/official && sha256sum -c SHA256SUMS.txt)
sha256sum docs/writeup/evidence/1020/tri_L9.npz   # vs SHA256.txt there
sha256sum <archive>.tar.zst                 # Drive manifest, column 3
python3 -I docs/writeup/calc/ledger_pairs_v1.py --chain 332675,333794,334806,335985,336415,336885,337303
--pairs 336885:337350,332675:337350
```

F.7 Data in Google Drive

Data too large for the repository is in the Drive folder arcwb-phase2-backup-20261021, shared on request; docs/writeup/evidence/drive_backup/DRIVE-BACKUP-MANIFEST-20261021.tsv lists each uploaded archive with sha256, md5 and size. §3 and Appendix C receipts (Monte Carlo truth, oracle and one-step runs) are in ext_arcwb_misc, the γ-check features (§2) in exp_claude_gpu_mc, and the learned parts' labels and training data in exp_claude_gpu_mc, exp_claude_liaison_local_20261017 and ext_arcwb_misc, without a manifest of network ids, seeds and commands. Drive's daily limit held back five archives on 21 October, among them the §3 ledger's chain states (ext_arcwb_r9A); they were uploaded on 22 October, each md5 matched, and the manifest lists all of them.

Appendix G. Diagnostics behind the claims

G.1 Measure the truth's own share first

A diagnostic found 93–95 % of the three-index error at layers 10–14 inside span(Q_c) (random-subspace baseline 0.41), seemingly blaming compression. But the truth itself lies 95–96 % inside it; the causal test, an oracle with exact old legs, gave a representation share of 0.06 (§3.3) [tuning networks]. **Rule:** measure a subspace share for the truth and for an innocent component before reading it as blame.

Sources: docs/BIGBET-OLDTIER-TRI-20261020.md.

G.2 Transfer between network sets

- Three decisions measured on 164 reused held-out networks kept 0.84–0.85 and 0.76 of their effect on new 320-network sets used once [held-out, used once], and 0.87 on a third reused under a logged budget [held-out, reused].
- These new-network expectations matched the official results at 1.04× (#337264), 1.05× (#337303) and 0.90× (#337350) [official bundle].
- Across 11 official records, new-network evidence carried over at 1.07 ± 0.10×, evidence from networks reused for selection at 0.29 ± 0.22× [official bundle].

Sources: `docs/writeup/SKELETON-1019-1020-ADDENDUM_v3.md` §9.2; `docs/FRESHEYES4-STATS-20261020.md`.

G.3 The one-step test

Table 3.3. One engine step with true inputs (§3.5; 64 random rows × all columns), without the engine's κ4→κ3 feed; energies in % of ‖T‖², half-split noise in brackets [tuning networks].

network, layer	new error	after true X	X's share, measured/noise-corrected
544, 10	0.45 (0.10)	0.11 (0.03)	0.74/0.76
544, 11	0.39 (0.07)	0.08 (0.02)	0.79/0.81
544, 12	0.42 (0.06)	0.09 (0.02)	0.79/0.80
544, 13	0.33 (0.04)	0.08 (0.01)	0.77/0.78
544, 14	0.38 (0.04)	0.08 (0.01)	0.78/0.79
624, 10	0.47 (0.11)	0.10 (0.04)	0.79/0.82
624, 12	0.46 (0.08)	0.09 (0.02)	0.81/0.84
624, 14	0.50 (0.05)	0.09 (0.01)	0.83/0.84

Consistency check, not a fit (§3.4). Let the standing error (% of ‖T‖²) follow $e_{\ell+1} = r \cdot e_\ell + s_\ell$, with $r \approx 0.8$, the transport share (76–86 %) if the error is carried like the cumulant. The growth curve implies s_ℓ = 0.37–0.53 % at layers 10–14. The test measures s = 0.33–0.50 %, whose plateau $s/(1-r)$ = 1.65–2.50 % is a relative error of 0.13–0.16, around the 0.140 at layer 14 [derivation].

Sources: `experiments/claude_ideas_20261014/onestep/README.md` §§1–4; `docs/writeup/v9/calc/depth_error_checks_v1.py` parts B–C.

Appendix H. Tool and scoring feedback: reproductions

Setup.

- *Releases:* flopscope 0.12.3 from PyPI (in-process), and flopscope-client and flopscope-server 0.12.3 built from the v0.12.3 tag. Also whestbench 0.16.2, NumPy 2.2.6 and Python 3.10, on one Linux machine.
- *Client–server mode:* through a Unix socket, as in flopscope's docker setup but without the hardened image.
- *Scripts:* each script in `feedback/` prints its own result, and `feedback/run_all.sh` runs them in both modes. Line numbers refer to 0.12.3.

H.1 (T1) A tuple `axis` fails in client–server mode

- **Repro.** `fnp.sum(fnp.ones((2, 3, 4)), axis=(0, 1))` returns shape (4,) in-process. In client–server mode it raises `TypeError: 'list' object cannot be interpreted as an integer.` `mean` and `max` behave the same, and an integer axis works.
- **Cause.** The client's `_encode_arg` sends tuples as lists (`flopscope-client/src/flopscope/_remote_array.py:1553`, "msgpack can't distinguish tuple/list"). The server decodes with `msgpack.unpackb` defaults, which give lists, and NumPy rejects a list axis.
- **Suggested fix.** Turn a list `axis` back into a tuple on the server, or decode with `use_list=False` [not tested].
- **Our case.** A derived correction stored a diagonal with `sum(axis=(0, 1, 3))`. In client–server mode it raised in every predict, and our guard returned its fallback answer, about 2000× worse. In-

process checks on 164 networks had passed. We caught it before submitting.

H.2 (T2) einsum's cost analysis allocates operand-sized float64 buffers

- **Sites.** Each builds `np.empty(shape)` in float64 for every operand, only to read shapes and object identity:
  - `_accumulation/_cost.py:532`, in `_walk_path_and_aggregate`, whose comment says "We use shapes=True to avoid materializing tensors";
  - `_accumulation/_cost.py:46`;
  - `_einsum.py:112`, in `_rebuild`;
  - `_opt_einsum/_contract.py:1304`.
- **Repro.** The operands are two float32 arrays of shape (8, 512, 512) and one of shape (8, 512).
  - NumPy's `einsum("kij,kij,kj->i")` peaks at +0.1 MiB. Flopscope's first call peaks at +32.3 MiB, and its cached second call at +8.0 MiB, the computation's own intermediate. With the patch the first call peaks at +8.2 MiB.
  - Under an address-space limit with 24 MiB of headroom, NumPy passes. Flopscope raises `MemoryError: Unable to allocate 16.0 MiB for an array with shape (8, 512, 512) and data type float64`.
  - Two distinct operands allocate nothing, and later calls with the same expression and shapes hit a cache.
- **In our runs.** Under the grader's 8 GiB limit, in our pre-submission check, it surfaced as `Unable to allocate 64.0 MiB … (8, 1024, 1024) float64`.
- **Patch** (`feedback/einsum-dummies.patch`). Replace each `np.empty(shape)` with `np.broadcast_to(np.empty(()), shape)`, which has the right shape, is a distinct object and takes 8 bytes. We tested it on six expressions, including repeated and symmetry-tagged operands:
  - FLOP counts and output bytes are identical.
  - Peak memory falls from 8.3 to 2.2 MiB, and with a repeated operand from 12.1 to 2.1 MiB.
  - flopscope's own test suite gives the same results with and without the patch (H.6).

H.3 (T3) `out=` returns a new object in client–server mode

- **Repro.** For `r = fnp.add(a, a, out=c)`, `r is c` is True in-process and False in client–server mode. Both name the same server array: a later write through `c` shows in `r`.
- **Effect.** Caches keyed on the returned object miss. Moving ours to the pool object saved 350–500 of about 47,700 requests per predict.

H.4 (T4) `weights.csv` in the wheel

- **What ships.** The 0.12.3 wheel ships `flopscope/data/weights.csv` next to `default_weights.json`.
- **The mismatch.** For 96 of the 461 operations that flopscope bills, its "Active Weight" column differs from the billed weight. Examples are `take`, `take_along_axis` and `choose` (0 against 4), `argsort` and `searchsorted` (1 against 4), and `hamming` (8 against 1).
- **What the docs say.** The data folder's README calls the CSV historical evidence and says `default_weights.json` is the only weights file shipped in the wheel.
- **Suggested fix.** Drop the CSV from the wheel, or rename the column.

H.5 (S1, S2) Failure and setup

- **Failed networks** (whestbench 0.16.2 `scoring.py`).
  - A failed network's predictions are zeroed, its multiplier is forced to 1.0, and the aggregate is the arithmetic mean. Zero predictions give a final-layer MSE of 0.7–1.2 on four public networks, so one failure adds about 0.01 to a score of 2e-9.
  - In grading it was harsher still. #337298 completed 67 networks; then one network hit the time limit (predict 108 s), the other 32 were not scored, and the submission was marked failed and not ranked.
- **Setup window** (whestbench 0.16.2 `runner.py`, `SubprocessRunner.start`).

- The 5.0 s timeout starts before the worker process is spawned, so it includes interpreter start and imports. A time-out raises `SETUP_TIMEOUT`, which fails the entire submission, as the CLI help says.
- `whest validate` times `estimator.setup()` alone, in-process.

- **Setup in grading.** Five workers start on one machine.
  - The setup of #337287 took 2.8 s alone on our machine. In grading it took 4.47–4.52 s on four workers, and the fifth went past 5 s. Across runs, workers' setups took 4.3–4.7 s, up to 0.7 s apart within one run.
  - Of our ~17 submissions with a median setup of 4.3 s or more, two failed with only four workers' setups recorded (#336511, #337287).

- **Compute** (S3). The official score is the final-layer MSE × max(0.1, F/B), which we checked on 50 official networks within 0.05 %. So a 1 % compute cut is worth exactly a 1 % accuracy gain.

H.6 Checking the patch

flopscope's tests ran from the v0.12.3 tag, stock and patched, with the same NumPy and opt_einsum 3.4.0.

- *einsum, contraction, accumulation and symmetry subset:* 1,008 passed, 1 failed and 2 skipped in both. The one failure, `test_two_operand_einsum_reaches_blas_speed`, is a timing test.
- *Full suite, serial (10,829 tests):* no test fails only with the patch. About 2,400 tests fail in both runs in our setup, for example with `Cannot nest BudgetContexts`; we did not investigate them. That timing test failed only in the stock run. [reproduced]

Sources: `feedback/` (scripts, patch, outputs); `experiments/claude_liaison_local_20261017/official_eval/eval_337298_fp3_failed_net66_timeout.json`, `eval_337287_V4R_failed.json`.